

CODING · AULA 8 · 6 DE OUTUBRO DE 2026

Loja de Roupas

Um projeto Python do zero até o GitHub, passando por tudo o que vimos: tipos, funções, testes, classes, herança, polimorfismo e contratos. Eu digito, vocês digitam junto.

Três regras para ninguém ficar para trás

REGRA 1

Digite, não copie

O botão Copiar existe para emergência. O que fica na cabeça é o que passou pelos dedos.

REGRA 2

Fase só acaba no verde

Cada fase termina com `pytest` passando e um commit no GitHub. Sem verde, não avançamos.

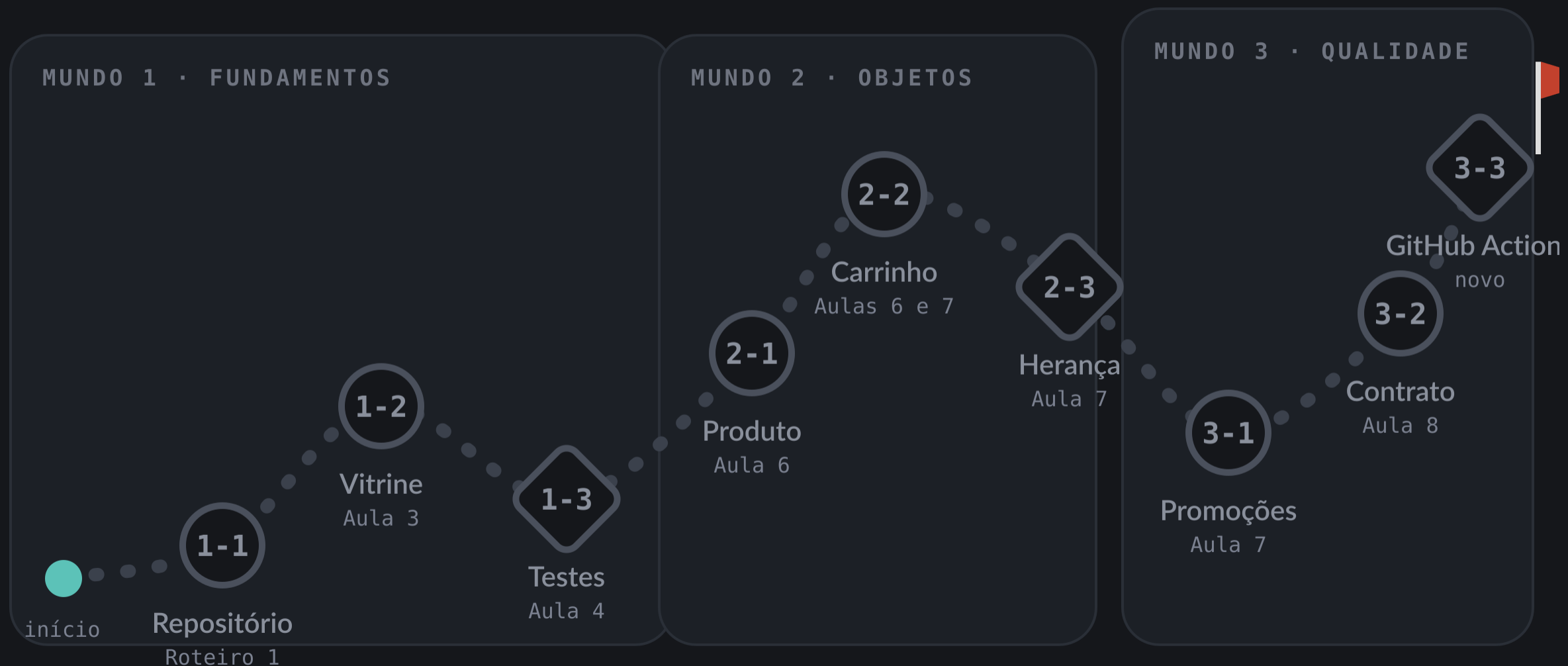
REGRA 3

Travou? Checkpoint

Toda fase tem um gabarito pronto no GitHub. Em 30 segundos você baixa e volta para o mesmo ponto da turma.

ATALHO

Aperte `M` a qualquer momento para ver o mapa e onde estamos.



O MAPA DE HOJE

Nove fases, três mundos

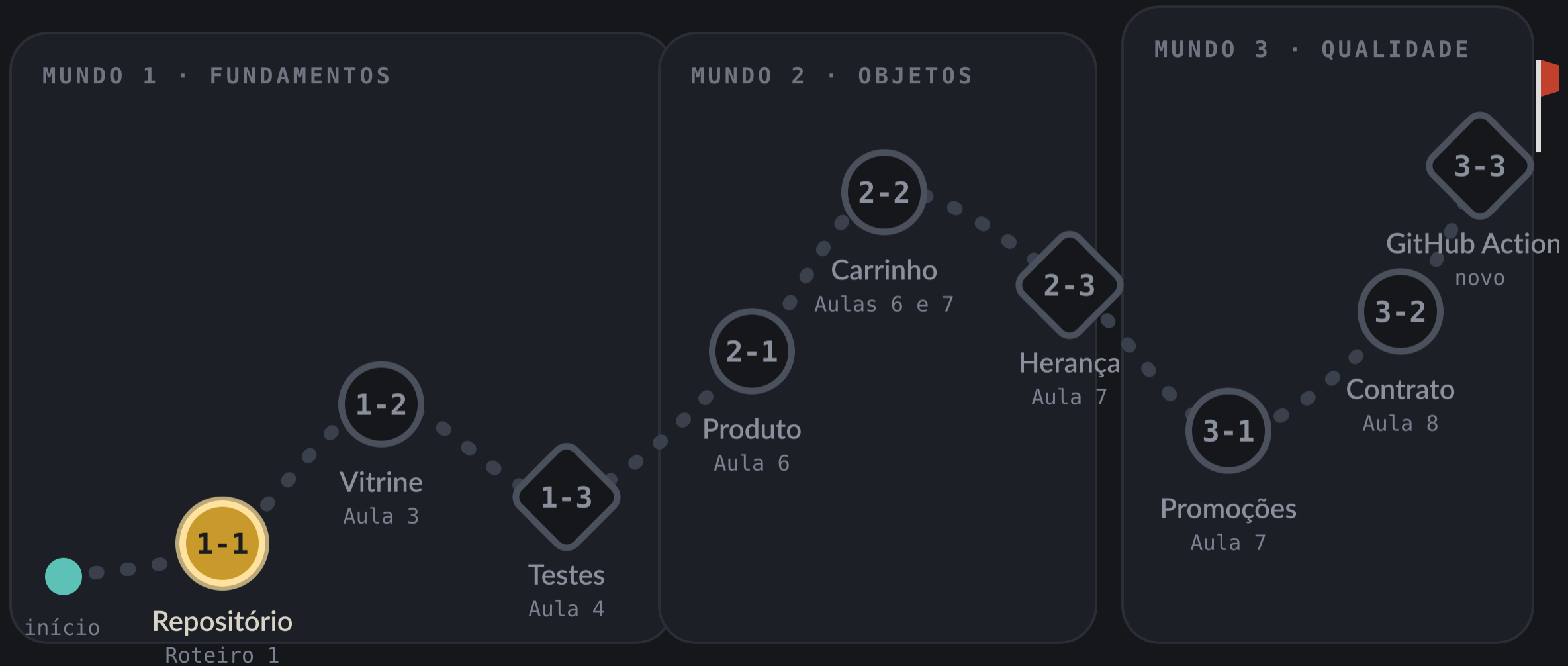
Cada fase revisita uma aula. As fases com losango são os chefes de cada mundo: as que mais pesam no projeto de vocês.

O QUE VAMOS CONSTRUIR

A loja, no fim do dia

```
loja-roupas/
├── .github/workflows/testes.yml  # o GitHub roda os testes
├── loja/
│   ├── calculos.py              # total e frete
│   ├── produto.py              # Produto, Camiseta, Calça
│   ├── carrinho.py              # o Carrinho
│   └── promocao.py              # o contrato e as promoções
├── tests/                       # 30 testes
├── vitrine.py                   # o primeiro script
├── requirements.txt
└── README.md
```

- ◆ Uma loja que vende **camisetas, calças e molettons** em tamanhos PP a GG.
- ◆ Um **carrinho** que soma as peças, cobra frete abaixo de R\$ 200 e aceita **promoções**.
- ◆ **30 testes** conferindo cada regra.
- ◆ Tudo no **GitHub**, com um ✓ verde a cada push.



FASE 1-1 · REVISITA ROTEIRO 1

O repositório

Antes de qualquer linha de Python: um lugar onde o código fica guardado, com histórico, e que a equipe inteira enxerga.

AO FINAL DA FASE

projeto no GitHub com README, .gitignore e requirements.txt

Repositório e commit

Repositório é a pasta do projeto com memória: o Git guarda cada versão salva. **Commit** é uma dessas versões, uma foto do projeto com uma mensagem dizendo o que mudou.

PENSE ASSIM

O histórico de versões de um documento, só que você decide quando tirar a foto e escreve a legenda.

CUIDADO

O commit fica só no seu computador até o `git push`.

exemplo

```
git log --oneline  
# 8da2ea2 Fase 2-1: classe Produto  
# 5bb8b16 Fase 1-3: calculos e testes  
# ed82f9b Fase 1-2: vitrine
```

Git e GitHub não são a mesma coisa



GIT

O histórico

Programa no seu computador. Cada commit é uma foto do projeto que dá para rever e desfazer.

GITHUB

A nuvem

Site que guarda uma cópia do repositório. É por ele que a equipe compartilha e o professor corrige.

Criar o repositório no GitHub

- ◆ Em github.com, clique em **New** (ou no + do canto superior).
- ◆ Repository name: `loja-roupas`. Deixe **Public**.
- ◆ Marque **Add a README file**. Clique em **Create repository**.
- ◆ Clique no botão verde **Code** e copie o endereço **HTTPS**.

POR QUE README JÁ NA CRIAÇÃO?

Assim o repositório nasce com um commit e o `git clone` funciona direto, sem configuração extra.

Trazer o repositório para o computador

```
terminal
$ git clone https://github.com/SEU-USUARIO/loja-roupas.git
Cloning into 'loja-roupas'...
$ cd loja-roupas
$ code .
# o VS Code abre na pasta do projeto
```

- ◆ `clone` baixa o repositório com todo o histórico e já liga a pasta ao GitHub.
- ◆ Troque `SEU-USUARIO` pelo seu usuário. Melhor ainda: cole o endereço copiado.
- ◆ Daqui para frente, use o terminal do VS Code: menu **Terminal › New Terminal**.

Ambiente virtual

Uma pasta, a `.venv`, com um Python e bibliotecas só deste projeto. Com ela ativada, o `pip install` instala ali dentro, sem mexer no resto do computador.

PENSE ASSIM

Uma mala por viagem: cada projeto leva só o que precisa, nas versões que precisa.

CUIDADO

Sem `(.venv)` no começo da linha, o pip instala no lugar errado e o `pytest` "some".

exemplo

```
python -m venv .venv          # cria
source .venv/bin/activate     # ativa
# Windows: .venv\Scripts\activate
pip install pytest            # instala só aqui
pip freeze                    # lista o que tem
```

O ambiente virtual e o pytest

```
terminal
$ python -m venv .venv
$ source .venv/bin/activate
# Windows: .venv\Scripts\activate
(.venv) aparece no começo da linha
$ pip install pytest
Successfully installed pytest-8...
$ echo pytest > requirements.txt
```

- ◆ O `.venv` é uma caixa de bibliotecas só deste projeto. Um projeto não quebra o outro.
- ◆ `requirements.txt` lista o que o projeto precisa. Quem clonar roda `pip install -r requirements.txt`.
- ◆ Se o `(.venv)` sumiu do terminal, ative de novo. É o erro mais comum do dia.

O que o Git deve ignorar



The screenshot shows a code editor window titled ".gitignore". The left sidebar displays a file tree for a project named "LOJA-ROUPAS", with files ".gitignore", "README.md", and "requirements.txt". The main editor area contains the following text:

```
.venv/  
__pycache__/  
.pytest_cache/  
*.pyc  
.DS_Store
```

- ◆ Crie o arquivo `.gitignore` na raiz, com o ponto no início do nome.
- ◆ `.venv/` é pesado e cada um cria o seu. **Nunca** vai para o GitHub.
- ◆ `__pycache__` e `.pytest_cache` são gerados sozinhos pelo Python e pelo pytest.

O ciclo que vamos repetir o dia inteiro

```
terminal
$ git status
Untracked files: .gitignore requirements.txt
$ git add .
$ git status
Changes to be committed: .gitignore requirements.txt
$ git commit -m "Fase 1-1: ambiente e gitignore"
$ git push
```

- ◆ `git status` é a pergunta "o que mudou?". Use sem medo, quantas vezes quiser.
- ◆ Vermelho no status: mudou e **não** está na foto. Verde: vai entrar no próximo commit.
- ◆ Na primeira vez, o `git push` abre o navegador para você entrar no GitHub.
- ◆ Repare: a `.venv` **não** apareceu no status. É o `.gitignore` trabalhando.

FASE 1-1 · CONCLUÍDA

Checkpoint: salvar e subir

```
checkpoint  
  
$ git add .  
$ git commit -m "Fase 1-1: ambiente e gitignore"  
$ git push
```

CONFIRA

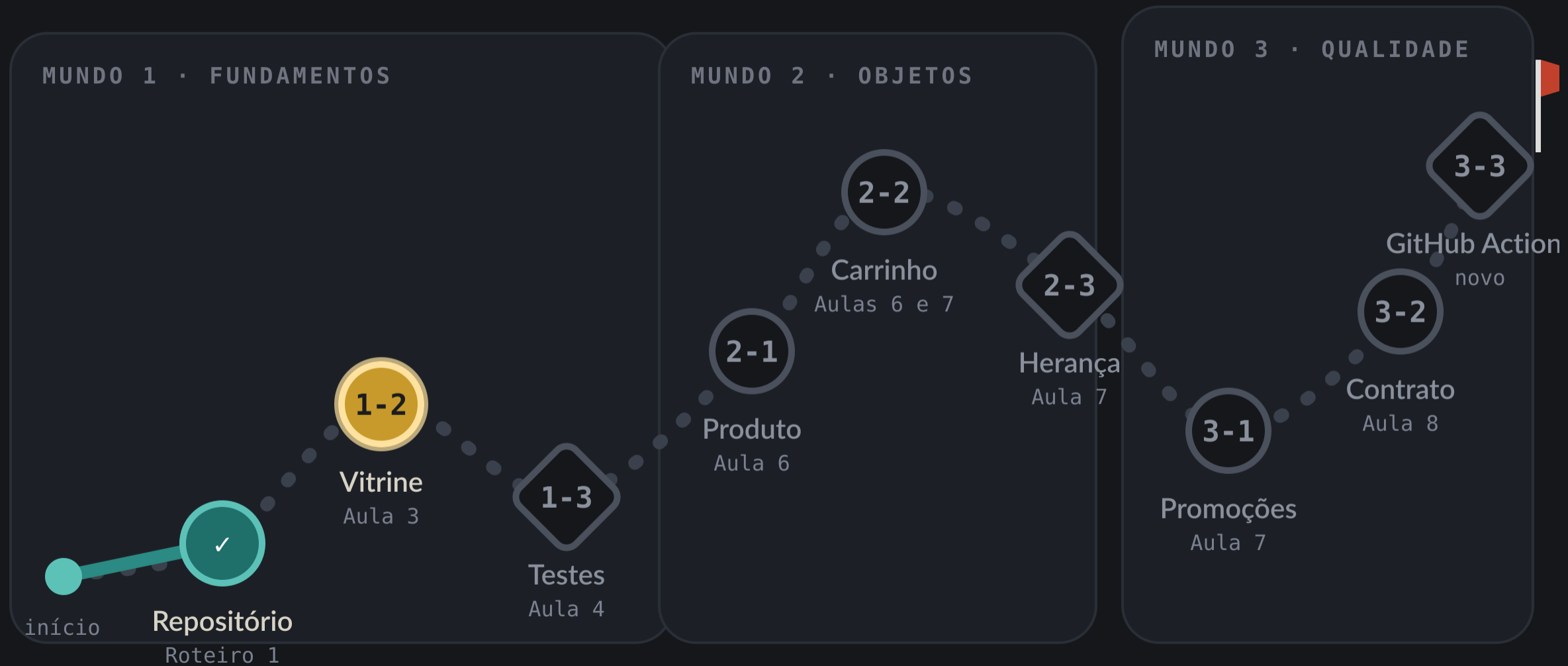
No GitHub, a página do repositório mostra README.md, .gitignore e requirements.txt, e não mostra .venv.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-01.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-01`



FASE 1-2 · REVISITA AULA 3

A vitrine

Os dados da loja em tipos e coleções: tupla, lista e dicionário, cada um com o seu papel.

AO FINAL DA FASE

um script que mostra quantas peças há na vitrine e o total do carrinho

Variável e tipo

Variável é um nome que aponta para um valor. Todo valor tem um **tipo**, que diz o que dá para fazer com ele.

CUIDADO

"3" é texto, não número: "3" * 2 dá "33".

exemplo

```
nome = "Camiseta básica"    # str: texto
quantidade = 3              # int: inteiro
preco = 39.90               # float: com casas decimais
em_estoque = True           # bool: verdadeiro ou falso

print(type(preco))          # <class 'float'>
```

Lista e tupla

As duas guardam vários valores em ordem, acessados pela **posição**, que começa em 0. A **lista** pode mudar; a **tupla**, não.

PENSE ASSIM

A lista é o carrinho: entra e sai peça. A tupla é a tabela de tamanhos: fixa.

CUIDADO

`TAMANHOS.append("GG")` dá erro: tupla não muda.

exemplo

```
carrinho = ["camiseta", "calça"]      # lista: colchetes
carrinho.append("moletom")           # cresce

TAMANHOS = ("P", "M", "G")           # tupla: parênteses

print(carrinho[0])                   # camiseta
print(TAMANHOS[-1])                  # G, o último
```

Dicionário

Guarda pares **chave e valor**. Em vez de achar pela posição, você acha pelo nome da chave.

PENSE ASSIM

A etiqueta da peça: em "preço" está 159,90; em "tamanho" está P.

CUIDADO

Chave que não existe dá `KeyError`:

```
produto["cor"]
```

exemplo

```
produto = {"nome": "Moletom", "preço": 159.90}
```

```
print(produto["preço"])      # 159.9
```

```
produto["tamanho"] = "P"    # chave nova
```

```
print(len(produto))         # 3
```

vitrine.py: os dados da loja

```
LOJA-ROUPAS
> .venv
.gitignore
README.md
requirements.txt
vitrine.py

# Etapa 2: os dados da loja em tipos e coleções (Aula 3)

# tupla: a lista de tamanhos não muda
TAMANHOS = ("PP", "P", "M", "G", "GG")

# lista de dicionários: um por produto
vitrine = [
    {"nome": "Camiseta básica", "preco": 39.90, "tamanho": "M"},
    {"nome": "Calça jeans", "preco": 129.90, "tamanho": "G"},
    {"nome": "Moletom", "preco": 159.90, "tamanho": "P"},
]

# lista de pares (nome, quantidade)
carrinho = [("Camiseta básica", 3), ("Calça jeans", 1)]
```

- ◆ Tupla **TAMANHOS**: uma sequência que não muda.
- ◆ Lista de dicionários: cada produto tem nome, preço e tamanho com nome.
- ◆ O carrinho é uma lista de pares **(nome, quantidade)**.

Laço for

Repete o mesmo bloco para cada **item** de uma coleção. Tudo o que está indentado embaixo do `for` roda uma vez por item.

CUIDADO

A indentação, 4 espaços, é o que diz o que está dentro do laço.

exemplo

```
for peca in ["camiseta", "calça"]:
    print("separando", peca)
# separando camiseta
# separando calça

for nome, qtd in [("camiseta", 3), ("calça", 1)]:
    print(nome, qtd)    # desempacota o par
```

vitrine.py: a conta do carrinho

```
LOJA-ROUPAS
> .venv
.gitignore
README.md
requirements.txt
vitrine.py

# dicionário: nome -> preço
precos = {}
for produto in vitrine:
    precos[produto["nome"]] = produto["preco"]

total = 0
for nome, quantidade in carrinho:
    total = total + precos[nome] * quantidade

print("Peças na vitrine:", len(vitrine))
print("Total do carrinho: R$", round(total, 2))
```

- ◆ O dicionário `precos` acha o preço pelo nome, sem procurar na lista toda.
- ◆ O segundo laço desempacota cada par: `nome, quantidade`.
- ◆ `round(total, 2)` só arredonda para mostrar. A conta continua em ponto flutuante.

Antes de rodar: o que aparece?

no terminal: python

```
print(vitrine[1]["preco"])
print(TAMANHOS[-1])
print(len(carrinho))
print(precos["Moletom"] * 2)
```

129.9: posição 1 da lista é a calça, e o zero à direita some.

GG: índice -1 é o último.

2: o carrinho tem dois pares, não quatro peças.

319.8: dois moletons.

Rodar o script

terminal

```
$ python vitrine.py
```

```
Peças na vitrine: 3
```

```
Total do carrinho: R$ 249.6
```

- ◆ Funciona. Mas e se o preço mudar? E se alguém escrever `+` no lugar de `*`?
- ◆ Para saber se continua certo, só rodando e **olhando com os olhos**. Toda vez.
- ◆ É esse o problema que a próxima fase resolve: fazer o computador conferir por nós.

FASE 1-2 · CONCLUÍDA

Checkpoint: salvar e subir

```
checkpoint  
  
$ git add .  
$ git commit -m "Fase 1-2: vitrine"  
$ git push
```

CONFIRA

O script imprime 3 peças e o total 249.6, e o vitrine.py aparece no GitHub.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-02.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-02`



FASE 1-3 · REVISITA AULA 4

Testes de verdade

O chefe do primeiro mundo. Vamos com calma: o que é um teste, o que o pytest faz, como ler quando dá errado e como escrever o teste antes do código.

AO FINAL DA FASE
4 testes passando e você sabendo ler um teste vermelho

FASE 1-3 · PARA COMEÇAR

Numa loja de roupa, ninguém vende a peça sem passar no provador.

Teste é o provador do código. Um programa pequeno que usa o seu programa e confere se a resposta está certa, sozinho, em milissegundos, quantas vezes você quiser.

Função

Um bloco com nome, que recebe valores (**parâmetros**) e devolve um resultado com **return**. Escreve uma vez, usa quantas vezes quiser.

PENSE ASSIM

Uma máquina: entra matéria-prima, sai produto. Quem usa não precisa saber como ela funciona por dentro.

CUIDADO

print só mostra na tela. **return** devolve para quem chamou. Só o que é devolvido pode ser testado.

exemplo

```
def preco_com_desconto(preco, pct):      # parâmetros
    return preco * (100 - pct) / 100     # devolve

valor = preco_com_desconto(100, 10)     # chamada
print(valor)                            # 90.0
```

Módulo e pacote

Módulo é um arquivo `.py`. Pacote é uma pasta de módulos com um `__init__.py`. O `import` traz para o seu arquivo o que está em outro.

CUIDADO

O `import` usa o caminho com pontos (`loja.calculos`), sem o `.py`.

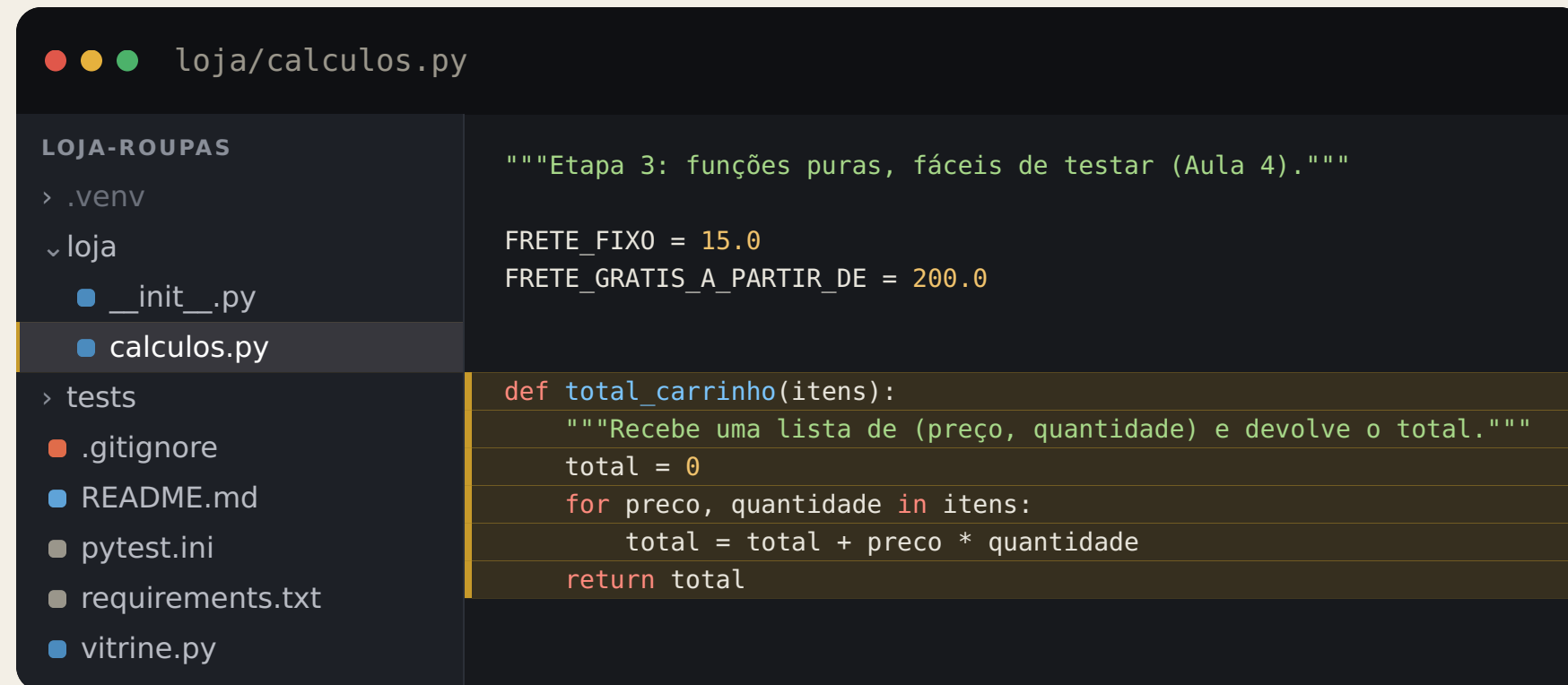
exemplo

```
# loja/calculos.py: módulo calculos, no pacote loja
```

```
from loja.calculos import total_carrinho
total_carrinho([(10, 2)])           # 20
```

```
import loja.calculos
loja.calculos.total_carrinho([])    # 0
```

Primeiro passo: a conta vira função



The screenshot shows a code editor with a dark theme. The top bar indicates the file path 'loja/calculos.py'. The left sidebar shows a file explorer for a project named 'LOJA-ROUPAS'. The explorer lists files: '.venv', 'loja' (expanded), '__init__.py', 'calculos.py' (selected), 'tests', '.gitignore', 'README.md', 'pytest.ini', 'requirements.txt', and 'vitrine.py'. The main editor area displays the code in 'calculos.py':

```
"""Etapa 3: funções puras, fáceis de testar (Aula 4)."""

FRETE_FIXO = 15.0
FRETE_GRATIS_A_PARTIR_DE = 200.0

def total_carrinho(itens):
    """Recebe uma lista de (preço, quantidade) e devolve o total."""
    total = 0
    for preco, quantidade in itens:
        total = total + preco * quantidade
    return total
```

- ◆ Crie a pasta `loja/` com um `__init__.py` **vazio**: ela vira um pacote que dá para importar.
- ◆ A função recebe a lista e **devolve** o total, sem `print`. O que é devolvido pode ser conferido.
- ◆ Função que só recebe e devolve, sem tela e sem arquivo, é a mais fácil de testar.

Um teste, sem pytest nenhum

no terminal: python

```
from loja.calculos import total_carrinho

assert total_carrinho([]) == 0          # verdadeiro: nada acontece
assert total_carrinho([(10, 2)]) == 20 # verdadeiro: nada acontece
assert total_carrinho([(10, 2)]) == 99 # falso: AssertionError
```

- ◆ `assert condição` não faz nada se a condição é verdadeira, e **explode** se é falsa.
- ◆ Todo teste é isso: rodar o código e afirmar o que deveria sair.
- ◆ O pytest só organiza: acha os asserts, roda todos e mostra o placar.

Todo teste tem três partes



Em inglês: Arrange, Act, Assert. Se faltar o 3, não é teste.

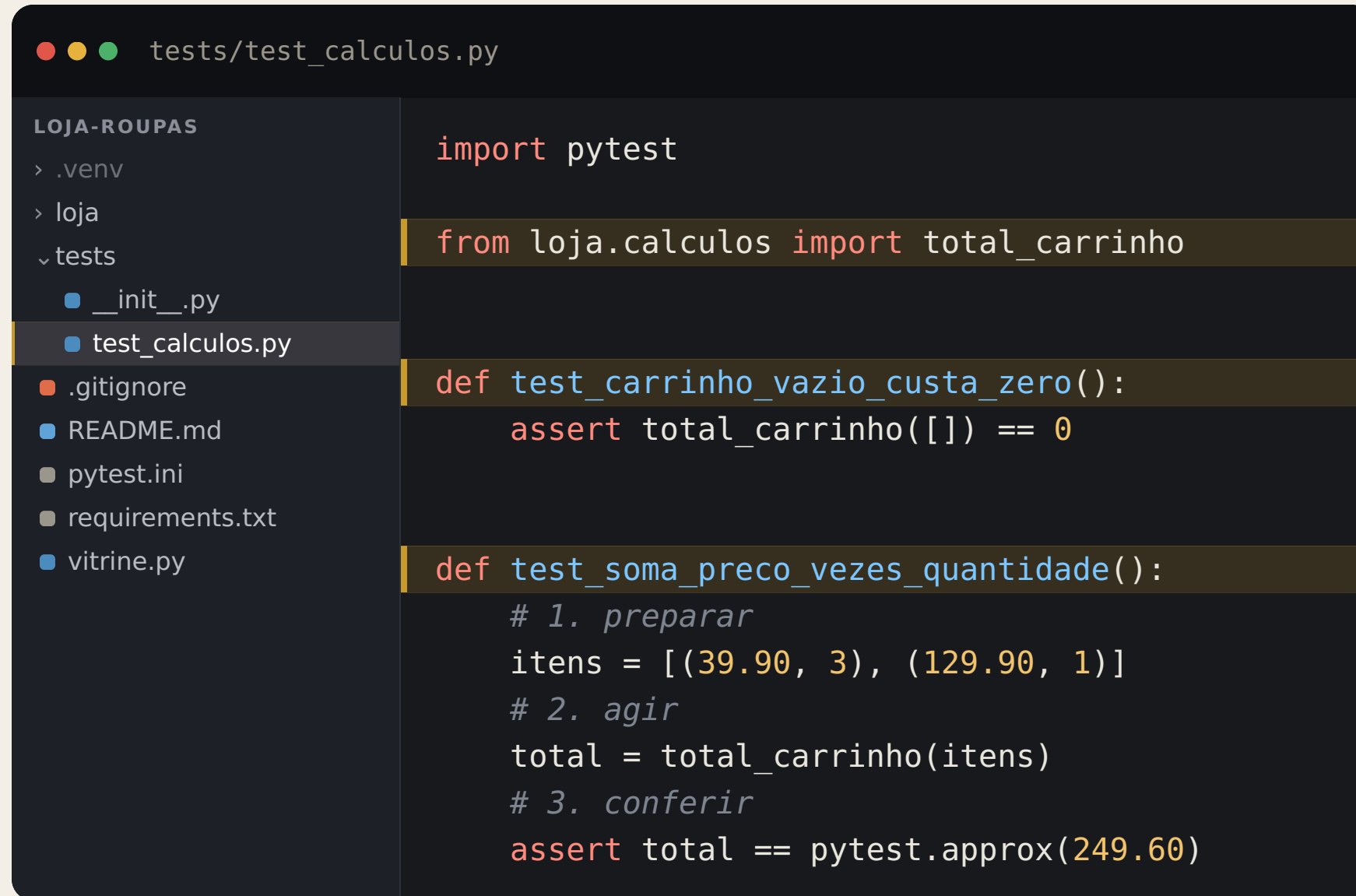
```
tests/test_calculos.py (um teste)
```

LOJA-ROUPAS

- > .venv
- > loja
- ▼ tests
 - __init__.py
 - test_calculos.py
- .gitignore
- README.md
- pytest.ini
- requirements.txt
- vitrine.py

```
def test_soma_preco_vezes_quantidade():  
    # 1. preparar  
    itens = [(39.90, 3), (129.90, 1)]  
    # 2. agir  
    total = total_carrinho(itens)  
    # 3. conferir  
    assert total == pytest.approx(249.60)
```

O arquivo de testes



The screenshot shows a code editor with a dark theme. On the left is a file explorer for a project named 'LOJA-ROUPAS'. It shows a directory structure with '.venv', 'loja', and 'tests'. The 'tests' directory is expanded, showing '__init__.py' and 'test_calculos.py', with 'test_calculos.py' selected. Below the directory, other files like '.gitignore', 'README.md', 'pytest.ini', 'requirements.txt', and 'vitrine.py' are listed. On the right, the content of 'test_calculos.py' is displayed. It starts with 'import pytest' and 'from loja.calculos import total_carrinho'. There are two test functions: 'test_carrinho_vazio_custa_zero()' which asserts that an empty cart is zero, and 'test_soma_preco_vezes_quantidade()' which prepares a list of items, calculates the total, and asserts it is approximately 249.60.

```
tests/test_calculos.py

LOJA-ROUPAS
> .venv
> loja
  tests
    __init__.py
    test_calculos.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

import pytest

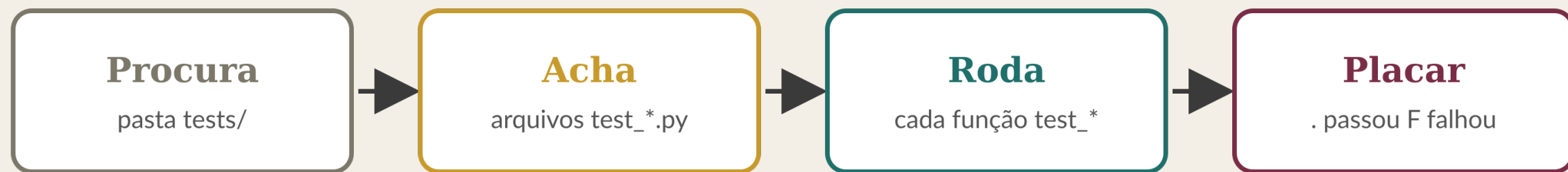
from loja.calculos import total_carrinho

def test_carrinho_vazio_custa_zero():
    assert total_carrinho([]) == 0

def test_soma_preco_vezes_quantidade():
    # 1. preparar
    itens = [(39.90, 3), (129.90, 1)]
    # 2. agir
    total = total_carrinho(itens)
    # 3. conferir
    assert total == pytest.approx(249.60)
```

- ◆ Pasta `tests/` com `__init__.py` vazio, e um `pytest.ini` na raiz para o pytest achar o pacote `loja`.
- ◆ O arquivo começa com `test_`. Cada função de teste também. É assim que o pytest acha.
- ◆ O nome do teste diz a regra: `test_carrinho_vazio_custa_zero`. Quando falhar, o nome já explica.

O que acontece quando você digita pytest



A screenshot of a code editor window titled 'pytest.ini'. The left sidebar shows a file tree for a project named 'LOJA-ROUPAS', with files like '.venv', 'loja', 'tests', '.gitignore', 'README.md', 'pytest.ini' (selected), 'requirements.txt', and 'vitrine.py'. The main editor area displays the following configuration:

```
[pytest]
pythonpath = .
testpaths = tests
```

PYTEST.INI

`pythonpath = .` deixa os testes importarem `loja`; `testpaths = tests` diz onde procurar.

Rodar: verde

```
terminal

$ pytest -v
collected 2 items

tests/test_calculos.py::test_carrinho_vazio_custa_zero PASSED [ 50%]
tests/test_calculos.py::test_soma_preco_vezes_quantidade PASSED [100%]
===== 2 passed in 0.01s =====
$ pytest
..
2 passed in 0.01s
```

- ◆ `-v` mostra o nome de cada teste. Sem ele, cada teste vira um ponto: `..`
- ◆ Repare no `pytest.approx(249.60)` do segundo teste. O próximo slide mostra por que ele está ali.

Por que o `pytest.approx`?

tests/test_ingenuo.py

```
def test_tres_camisetas():  
    assert total_carrinho([(39.90, 3)]) == 119.70
```

terminal

```
$ pytest  
F  
____ test_tres_camisetas ____  
> assert total_carrinho([(39.90, 3)]) == 119.70  
E      assert 119.69999999999999 == 119.7  
tests/test_ingenuo.py:5: AssertionError  
1 failed in 0.02s
```

- 1 O F e o nome do teste dizem **qual** regra quebrou.
- 2 A linha com `>` mostra **onde**: arquivo e linha.
- 3 As linhas com `E` mostram **o que saiu** contra **o que se esperava**.
- 4 Aqui o código está certo: é a Aula 3, ponto flutuante não é exato. Por isso `pytest.approx`.

Um teste só vale se já viu o erro

loja/calculos.py · com um erro de digitação

```
total = total + preco + quantidade # era *
```

terminal

```
$ pytest
```

```
.F
```

```
E      assert 173.8 == 249.6 ± 2.5e-04
```

```
E          Obtained: 173.8
```

```
E          Expected: 249.6 ± 2.5e-04
```

```
1 failed, 1 passed in 0.02s
```

- 1 Troque o `*` por `+` e rode. O teste pega na hora.
- 2 O `vitrine.py` também imprimiria 173.8, sem reclamar. Só alguém atento perceberia.
- 3 Desfaça o erro e rode de novo: verde. Um teste que nunca falhou não provou nada.

Ele passa. Mas testa alguma coisa?



```
tests/test_duvidoso.py

LOJA-ROUPAS
> .venv
> loja
  tests
    __init__.py
    test_calculos.py
    test_duvidoso.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

def test_total():
    total_carrinho([(39.90, 3)])
```

Não testa nada. Não tem `assert`:
chama a função e não confere o
resultado.

Ele passaria mesmo se a função
devolvesse 0, ou -1000. Só falharia
se a função levantasse um erro.

Regra: todo teste termina
conferindo alguma coisa.

FASE 1-3 · TESTE PRIMEIRO

Regra nova da loja: frete grátis a partir de R\$ 200. Abaixo disso, R\$ 15.

Desta vez vamos escrever o teste antes do código. Vermelho primeiro, depois o código, depois verde. O teste vira a especificação da regra.

Os testes da regra, antes da função existir

```
tests/test_calculos.py (fim do arquivo)

LOJA-ROUPAS
> .venv
> loja
  tests
    __init__.py
    test_calculos.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

def test_frete_abaixo_de_200_custa_15():
    assert frete(199.99) == 15.0

def test_frete_a_partir_de_200_e_gratis():
    assert frete(200.00) == 0.0
    assert frete(350.00) == 0.0
```

- ◆ Atualize o import: `from loja.calculos import frete, total_carrinho`.
- ◆ Um teste para cada lado da regra, e um para a **fronteira**: exatamente 200.
- ◆ A fronteira é onde mais se erra: `>` ou `>=`? O teste decide, não a memória.

Condição: if e else

Escolhe um caminho conforme uma comparação dê **verdadeiro** ou **falso**.

CUIDADO

`>` e `>=` só dão resultados diferentes exatamente na fronteira, em 200. É lá que o teste precisa olhar.

exemplo

```
valor = 200
```

```
if valor >= 200:  
    frete = 0
```

```
else:  
    frete = 15
```

```
print(frete)    # 0
```

Vermelho, vermelho, verde

1 · VERMELHO

Teste escrito, função ainda não existe

2 · AINDA VERMELHO

Primeira versão, com >

3 · VERDE

Corrigida, com >=

1 erro

terminal

```
$ pytest
ERROR tests/test_calculos.py
E   ImportError: cannot import name 'frete' from 'loja.calculos'
Interrupted: 1 error during collection
```

LENDO

Não é falha de regra: o teste nem conseguiu importar. É o esperado, a função ainda não existe.

Vermelho, vermelho, verde

1 · VERMELHO

Teste escrito, função ainda não existe

2 · AINDA VERMELHO

Primeira versão, com >

3 · VERDE

Corrigida, com >=

1 failed, 3 passed

loja/calculos.py

LOJA-ROUPAS

> .venv

✓ loja

■ __init__.py

■ calculos.py

> tests

■ .gitignore

■ README.md

■ pytest.ini

■ requirements.txt

■ vitrine.py

```
def frete(valor_da_compra):  
    if valor_da_compra > FRETE_GRATIS_A_PARTIR_DE:  
        return 0.0  
    return FRETE_FIXO
```

terminal

```
$ pytest
```

```
>         assert frete(200.00) == 0.0
```

```
E         assert 15.0 == 0.0
```

```
1 failed, 3 passed in 0.01s
```

Vermelho, vermelho, verde

1 · VERMELHO

Teste escrito, função ainda não existe

2 · AINDA VERMELHO

Primeira versão, com >

3 · VERDE

Corrigida, com >=

4 passed

loja/calculos.py

LOJA-ROUPAS

> .venv
v loja
 __init__.py
 calculos.py
> tests
 .gitignore
 README.md
 pytest.ini
 requirements.txt
 vitrine.py

```
FRETE_FIXO = 15.0
FRETE_GRATIS_A_PARTIR_DE = 200.0

def frete(valor_da_compra):
    """Frete grátis a partir de R$ 200; abaixo disso, R$ 15."""
    if valor_da_compra >= FRETE_GRATIS_A_PARTIR_DE:
        return 0.0
    return FRETE_FIXO
```

terminal

```
$ pytest
```

```
.....
```

```
4 passed in 0.01s
```

Como escrever um bom teste

REGRA	NA LOJA
O nome do teste diz a regra	<code>test_frete_a_partir_de_200_e_gratis</code>
Preparar, agir, conferir	itens, <code>total_carrinho(itens)</code> , <code>assert</code>
Um comportamento por teste	frete abaixo de 200 e a partir de 200 em testes separados
Teste a fronteira	exatamente R\$ 200,00
Dinheiro com casas decimais	<code>pytest.approx</code>
Veja o teste falhar uma vez	troque <code>*</code> por <code>+</code> e rode

FASE 1-3 · CONCLUÍDA

Checkpoint: salvar e subir

```
checkpoint  
  
$ pytest  
4 passed  
$ git add .  
$ git commit -m "Fase 1-3: calculos e primeiros testes"  
$ git push
```

CONFIRA

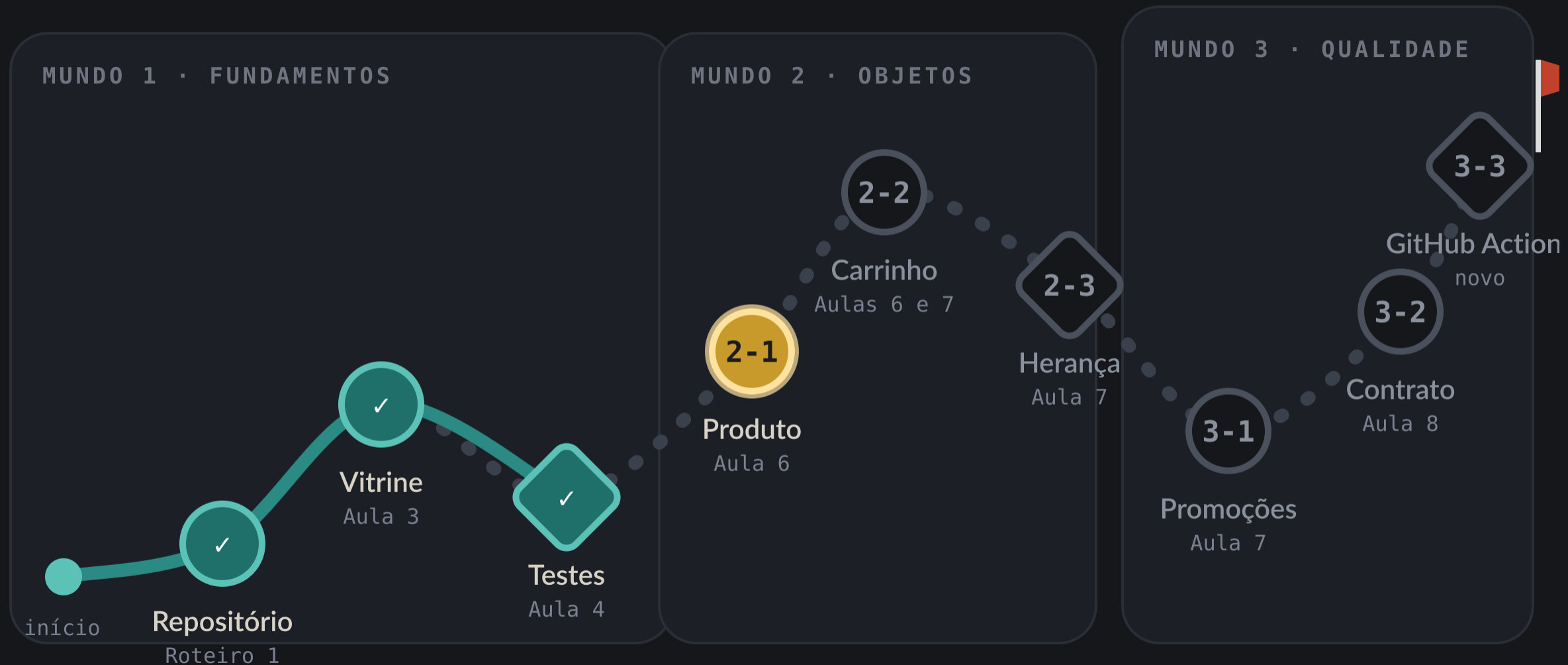
O `pytest` mostra 4 passed e a pasta loja/ e a tests/ aparecem no GitHub.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-03.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-03`



FASE 2-1 · REVISITA AULA 6

O produto vira classe

No dicionário, qualquer coisa entra. Na classe, o produto se recusa a nascer errado.

AO FINAL DA FASE

8 testes passando, incluindo os que esperam erro

Classe e objeto

Classe é o molde; **objeto** é cada peça feita com ele. A classe junta os dados (**atributos**) e o que dá para fazer com eles (**métodos**).

PENSE ASSIM

A ficha técnica da camiseta é a classe. Cada camiseta no estoque é um objeto.

CUIDADO

Cada objeto tem os próprios atributos: mudar

`camiseta.preco` não muda

`moletom.preco`.

exemplo

```
class Produto:                                # o molde
    ...

camiseta = Produto("Básica", 39.90, "M")      # um objeto
moletom = Produto("Moletom", 159.90, "P")    # outro

print(camiseta.nome)                          # atributo
print(camiseta.descricao())                  # método
```

Dicionário aceita tudo; classe confere

DICIONÁRIO

Entra qualquer coisa

```
● ● ● vitrine.py  
  
{"nome": "", "preco": -10, "tamanho": "XG"}
```

Nome vazio, preço negativo, tamanho que não existe.
Nada reclama.

CLASSE

Valida na porta de entrada

```
● ● ● loja/produto.py  
  
Produto("", -10, "XG")  
# ValueError: nome do produto não pode ser vazio
```

O `__init__` confere antes de guardar. Produto inválido nem chega a existir.

É a invariante da Aula 6: uma regra que vale **sempre**, garantida no ponto de entrada.

`__init__` e `self`

`__init__` roda sozinho quando o objeto nasce e prepara os atributos. `self` é o próprio objeto que está sendo criado ou usado.

CUIDADO

Sem o `self.`, a variável some quando o `__init__` termina.

exemplo

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome          # guarda no objeto
        self.preco = preco

p = Produto("Boné", 35)
# o Python chama __init__(p, "Boné", 35)
```

Exceção: raise

`raise` interrompe o programa e avisa que algo está errado, com um **tipo de erro** e uma **mensagem**.

CUIDADO

`ValueError`: valor inválido. `TypeError`: tipo errado. O tipo certo ajuda quem lê o erro.

exemplo

```
if preco <= 0:  
    raise ValueError("preço deve ser maior que zero")
```

no terminal:

ValueError: preço deve ser maior que zero

Invariante

Uma regra que precisa valer **sempre**, do nascimento ao fim do objeto. É conferida na porta de entrada, para nenhum objeto inválido existir.

CUIDADO

Validar só na tela não basta: a classe precisa se proteger sozinha.

exemplo

```
Produto:  preço > 0 e tamanho em TAMANHOS  
Carrinho: quantidade > 0  
          finalizado não recebe peças  
Promoção: total nunca negativo
```

loja/produto.py

```
loja/produto.py

LOJA-ROUPAS
> .venv
└─ loja
   ├── __init__.py
   ├── calculos.py
   └── produto.py
> tests
├── .gitignore
├── README.md
├── pytest.ini
├── requirements.txt
└── vitrine.py

"""Etapa 4: o produto vira classe (Aula 6)."""

TAMANHOS = ("PP", "P", "M", "G", "GG")

class Produto:
    def __init__(self, nome, preco, tamanho):
        if not nome or not nome.strip():
            raise ValueError("nome do produto não pode ser vazio")
        if preco <= 0:
            raise ValueError("preço deve ser maior que zero")
        if tamanho not in TAMANHOS:
            raise ValueError(f"tamanho inválido: {tamanho}")
        self.nome = nome.strip()
        self.preco = preco
        self.tamanho = tamanho

    def descricao(self):
        return f"{self.nome} {self.tamanho}: R$ {self.preco:.2f}"
```

- ◆ Três regras, conferidas **antes** de guardar qualquer atributo.
- ◆ `raise ValueError` interrompe a criação com uma mensagem que explica o problema.
- ◆ `descricao()` é comportamento junto do dado: o produto sabe se apresentar.

pytest.raises: passa quando o erro acontece

```
tests/test_produto.py (os dois primeiros)

LOJA-ROUPAS
> .venv
> loja
  tests
    ├── __init__.py
    ├── test_calculos.py
    └── test_produto.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

import pytest

from loja.produto import Produto

def test_produto_valido():
    camiseta = Produto("Camiseta básica", 39.90, "M")
    assert camiseta.descricao() == "Camiseta básica M: R$ 39.90"

def test_preco_zero_nao_e_aceito():
    with pytest.raises(ValueError):
        Produto("Camiseta básica", 0, "M")
```

- ◆ `with pytest.raises(ValueError):`
lê-se "o que vem a seguir tem que dar ValueError".
- ◆ Se o erro **não** acontecer, o teste falha. É assim que se prova que a regra existe.
- ◆ Teste o caminho feliz e cada regra que recusa.

Quais linhas levantam ValueError?

no terminal: python

```
a = Produto("Moletom", 159.90, "g")
b = Produto("  Boné  ", 35, "M")
c = Produto("Meia", 0, "P")
d = Produto("Jaqueta", 299.90, "GG")
```

a: ValueError. `"g"` minúsculo não está na tupla `TAMANHOS`.

b: nasce. O `strip()` guarda o nome como `"Boné"`.

c: ValueError. Preço tem que ser maior que zero.

d: nasce.

FASE 2-1 • CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
8 passed
$ git add .
$ git commit -m "Fase 2-1: classe Produto"
$ git push
```

CONFIRA

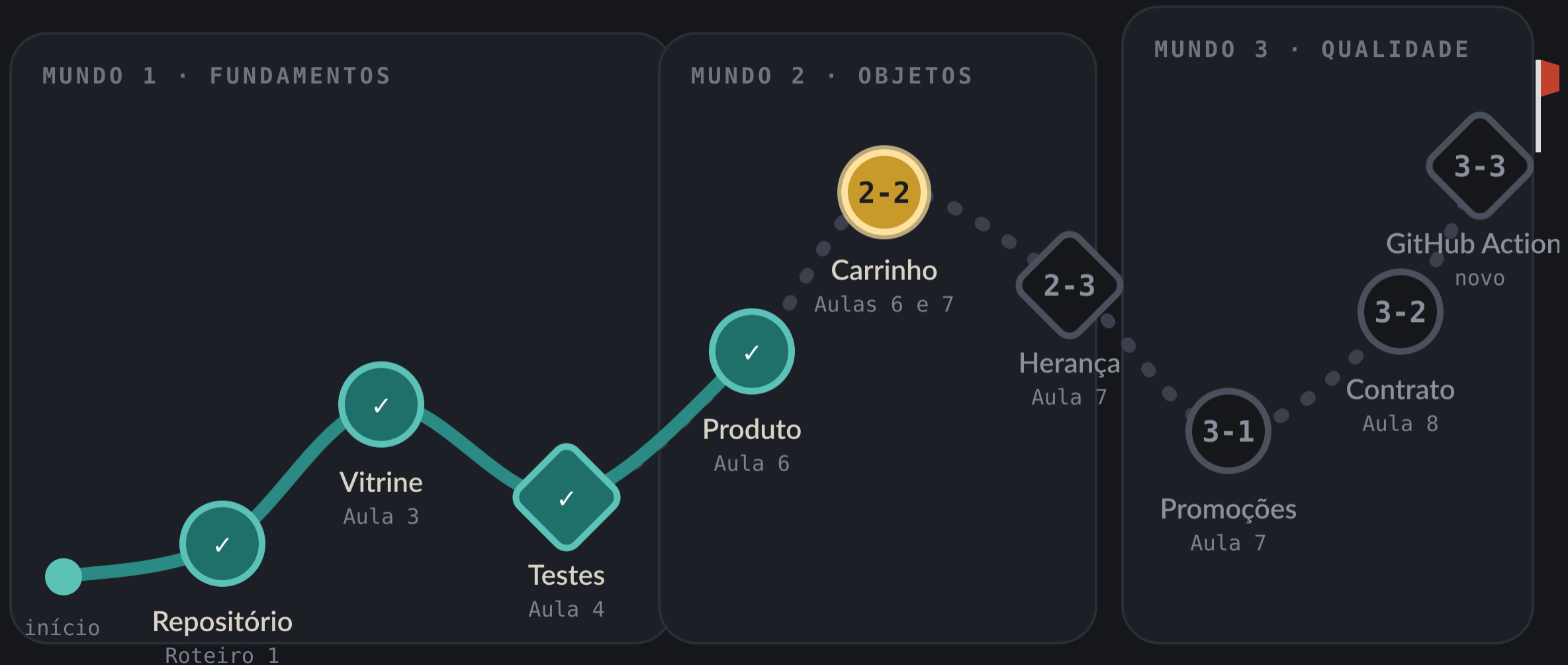
O `pytest` mostra **8 passed**: 4 de cálculo e 4 de produto.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-04.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-04`



FASE 2-2 · REVISITA AULAS 6 E 7

O carrinho

O carrinho não é um produto: ele TEM produtos. Composição, encapsulamento e propriedades.

AO FINAL DA FASE

15 testes passando, com o carrinho protegendo a própria lista

Composição

Uma classe que **tem** outra dentro de si e usa o que ela sabe fazer. É a relação "**tem um**".

PENSE ASSIM

O carrinho tem peças. Ele não é uma peça.

CUIDADO

Se a frase "X é um Y" soa estranha, é composição, não herança.

exemplo

```
class Carrinho:
    def __init__(self):
        self._itens = []    # o carrinho TEM uma lista

    def adicionar(self, produto, quantidade=1):
        self._itens.append((produto, quantidade))
```

Encapsulamento

Esconder o detalhe interno e oferecer **portas controladas**. Em Python, o `__` no começo do nome avisa: não mexa direto, use os métodos.

CUIDADO

O `__` é um combinado, não um cadeado. O cadeado é a validação dentro do método.

exemplo

```
c._itens.append(("x", -5))    # possível, mas fura a regra  
c.adicionar(camiseta, 2)     # a porta certa: valida antes
```

isinstance

Pergunta se um objeto é de uma classe, ou de uma filha dela. Devolve `True` ou `False`.

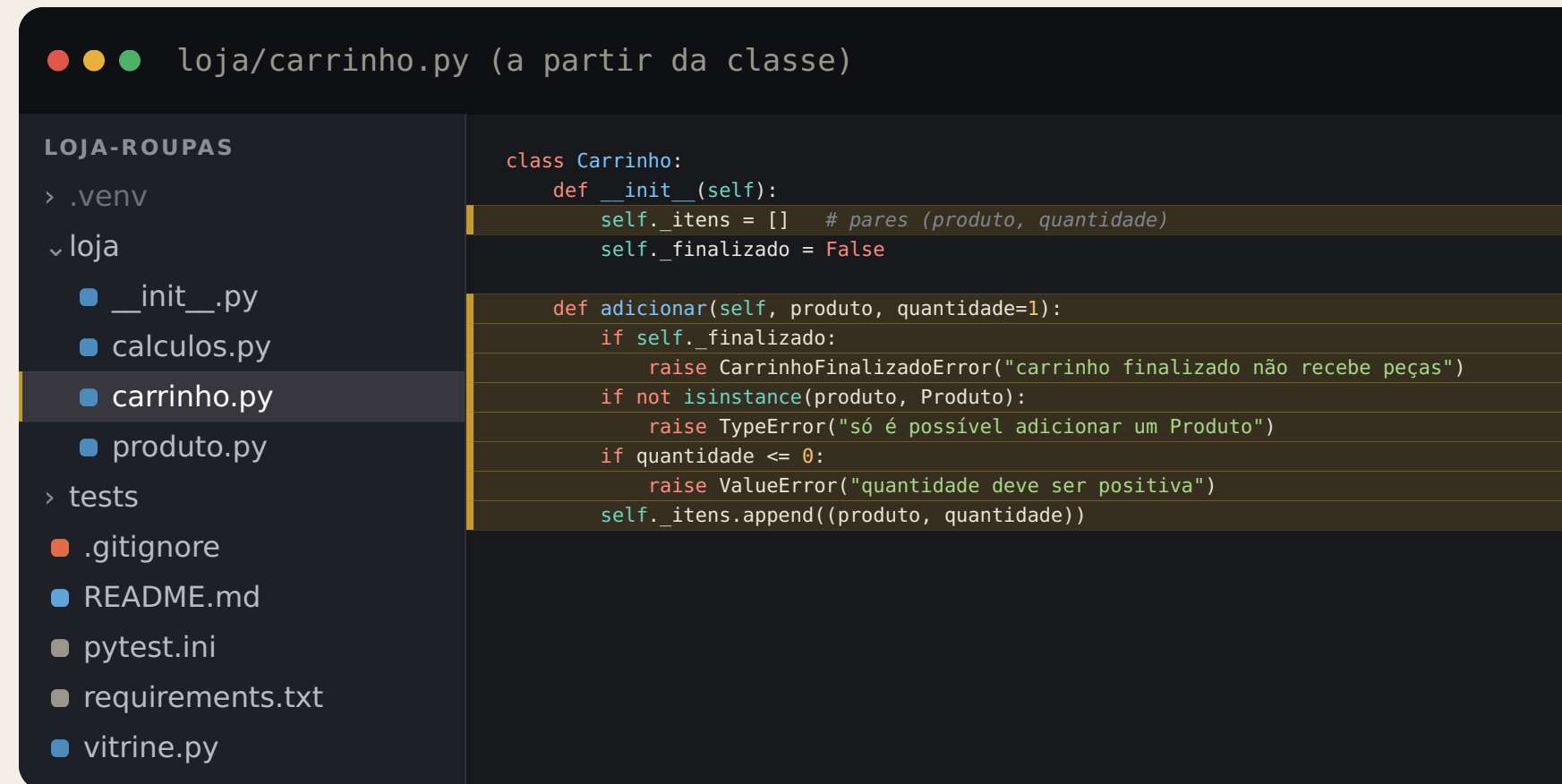
PENSE ASSIM

O segurança da loja conferindo o crachá antes de deixar entrar.

exemplo

```
isinstance(camiseta, Produto)           # True
isinstance(("Boné", 35), Produto)       # False: é uma tupla
isinstance(3, int)                       # True
```

loja/carrinho.py: a porta de entrada



```
loja/carrinho.py (a partir da classe)

class Carrinho:
    def __init__(self):
        self._itens = [] # pares (produto, quantidade)
        self._finalizado = False

    def adicionar(self, produto, quantidade=1):
        if self._finalizado:
            raise CarrinhoFinalizadoError("carrinho finalizado não recebe peças")
        if not isinstance(produto, Produto):
            raise TypeError("só é possível adicionar um Produto")
        if quantidade <= 0:
            raise ValueError("quantidade deve ser positiva")
        self._itens.append((produto, quantidade))
```

- ◆ `self._itens` guarda pares `(produto, quantidade)`. O underscore avisa: não mexa direto.
- ◆ `adicionar` é a única porta. Ela confere três regras antes do `append`.
- ◆ `CarrinhoFinalizadoError` é uma exceção nossa, com nome que explica o problema.

@property

Transforma um método em algo que se lê **como atributo**, sem parênteses. O valor é calculado na hora em que é lido.

CUIDADO

Por ser calculado na hora, o valor nunca fica desatualizado.

exemplo

```
class Carrinho:
    @property
    def quantidade_de_pecas(self):
        return sum(q for _, q in self._itens)

c.quantidade_de_pecas      # sem ()
```

carrinho.py: o que ele sabe responder

```
loja/carrinho.py (continuação)

LOJA-ROUPAS
> .venv
  loja
    __init__.py
    calculos.py
    carrinho.py
    produto.py
  tests
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

@property
def itens(self):
    return list(self._itens)  # uma cópia

@property
def quantidade_de_pecas(self):
    return sum(quantidade for _, quantidade in self._itens)

@property
def subtotal(self):
    return total_carrinho([(p.preco, q) for p, q in self._itens])

@property
def total(self):
    return self.subtotal + frete(self.subtotal)

def finalizar(self):
    if not self._itens:
        raise ValueError("não é possível finalizar um carrinho vazio")
    self._finalizado = True
```

- ◆ `@property` deixa escrever `c.subtotal`, sem parênteses, como se fosse atributo.
- ◆ `itens` devolve uma **cópia**: quem está de fora lê, mas não altera.
- ◆ `subtotal` reaproveita a função da Fase 1-3. O que já tem teste, a gente reusa.

Uma função que prepara, vários testes que conferem

```
tests/test_carrinho.py (trecho)

LOJA-ROUPAS
> .venv
> loja
  tests
    __init__.py
    test_calculos.py
    test_carrinho.py
    test_produto.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

def carrinho_exemplo():
    c = Carrinho()
    c.adicionar(Produto("Camiseta básica", 39.90, "M"), 3)
    c.adicionar(Produto("Calça jeans", 129.90, "G"))
    return c

def test_subtotal_e_quantidade():
    c = carrinho_exemplo()
    assert c.quantidade_de_pecas == 4
    assert c.subtotal == pytest.approx(249.60)

def test_acima_de_200_frete_gratis():
    assert carrinho_exemplo().total == pytest.approx(249.60)
```

- ◆ `carrinho_exemplo()` faz a parte preparar uma vez só. Cada teste chama e confere.
- ◆ Os dois testes de total cobrem os dois lados do frete: grátis e pago.
- ◆ No arquivo completo há mais 4 testes, um para cada regra de `adicionar` e para a cópia.

Checkpoint: salvar e subir

```
checkpoint  
  
$ pytest  
15 passed  
$ git add .  
$ git commit -m "Fase 2-2: Carrinho"  
$ git push
```

CONFIRA

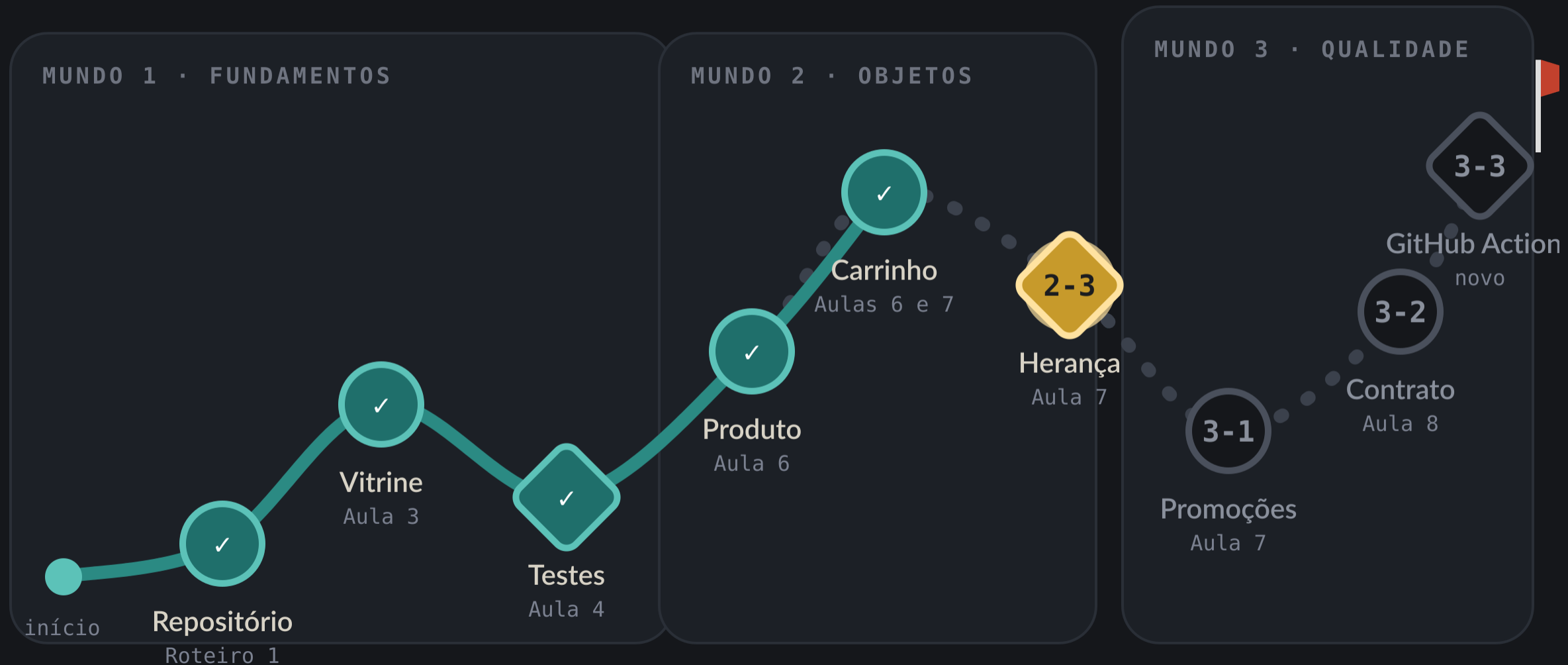
O `pytest` mostra **15 passed**. Teste no terminal: `c.itens.append("x")` não muda o carrinho.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-05.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-05`



FASE 2-3 · REVISITA AULA 7

Camiseta é um Produto

Herança só onde a frase "é um" soa natural, com `super()` preservando as regras.

AO FINAL DA FASE

19 testes passando, com a validação da mãe valendo para as filhas

Herdar ou compor?

É UM

Camiseta é um Produto

Tem nome, preço, tamanho e as mesmas regras.
Acrescenta só a manga. **Herança**.

TEM UM

Carrinho não é um Produto

Carrinho tem produtos, mas não tem tamanho nem preço próprio. **Composição**, como na Fase 2-2.

Herança

Uma classe **filha** recebe tudo da **mãe** e acrescenta ou muda só o que é dela. É a relação "é um".

CUIDADO

Herde só quando a frase "a filha é uma mãe" for verdadeira.

exemplo

```
class Camiseta(Produto):      # Camiseta é um Produto
    ...

c = Camiseta("Básica", 39.90, "M", "curta")
c.preco                       # veio de Produto
c.descricao()                 # veio de Produto
```

super()

Chama a versão da classe **mãe** de dentro da filha. No `__init__`, garante que a parte herdada nasça com as regras da mãe.

CUIDADO

Sem o `super().__init__`, a camiseta nasce sem nome, sem preço e sem validação.

exemplo

```
def __init__(self, nome, preco, tamanho, manga):  
    super().__init__(nome, preco, tamanho)    # a mãe valida  
    self.manga = manga                        # a filha acrescenta
```

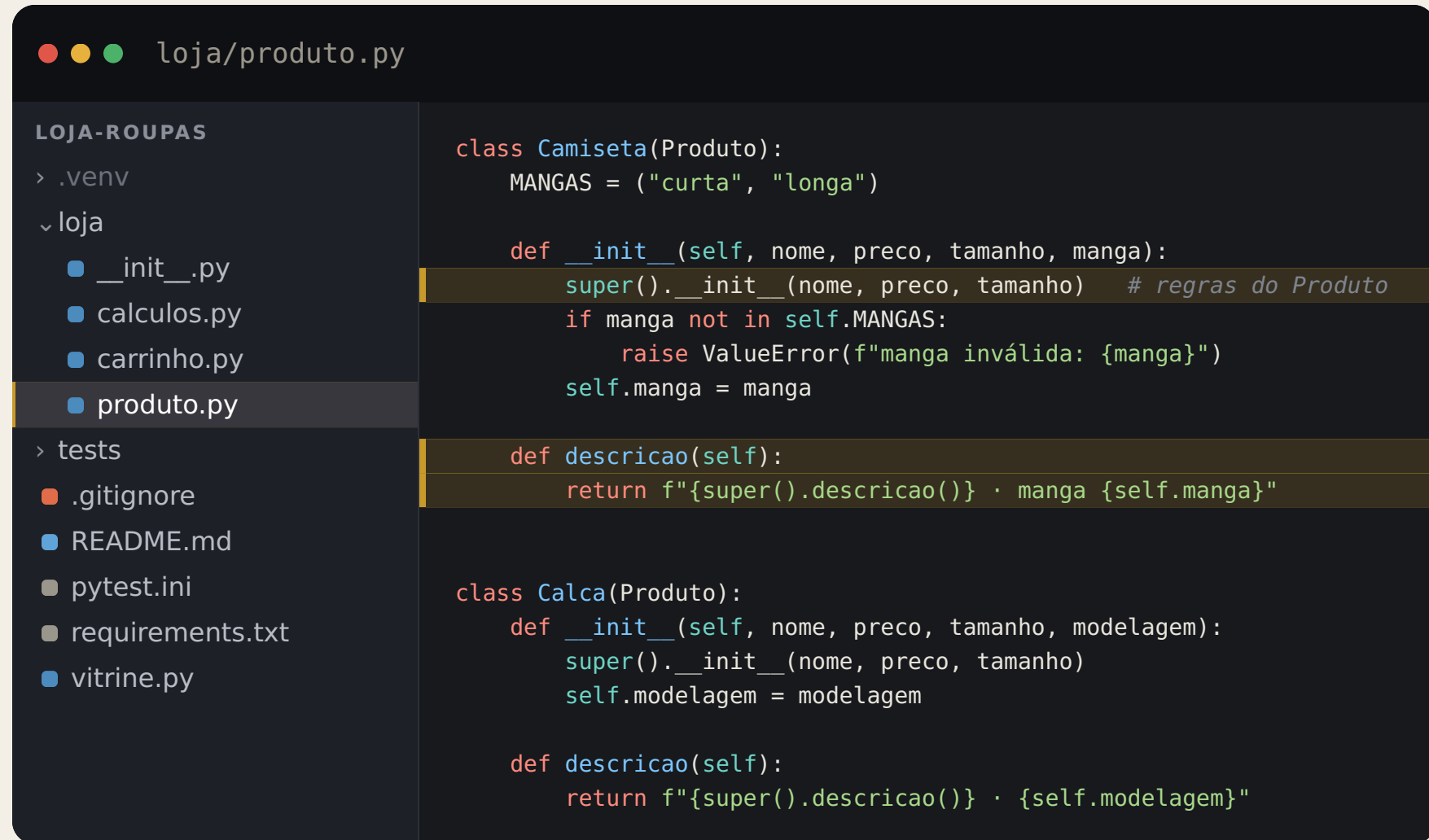
Sobrescrita

A filha **redefine** um método da mãe, com o mesmo nome. Quem chama não muda nada; muda o que acontece.

exemplo

```
class Calca(Produto):  
    def descricao(self):                # mesmo nome  
        base = super().descricao()    # reaproveita a mãe  
        return f"{base} · {self.modelagem}"
```

Camiseta e Calça (fim de loja/produto.py)



```
loja/produto.py

LOJA-ROUPAS
> .venv
└─ loja
   ├── __init__.py
   ├── calculos.py
   ├── carrinho.py
   └── produto.py
└─ tests
   ├── .gitignore
   ├── README.md
   ├── pytest.ini
   ├── requirements.txt
   └── vitrine.py

class Camiseta(Produto):
    MANGAS = ("curta", "longa")

    def __init__(self, nome, preco, tamanho, manga):
        super().__init__(nome, preco, tamanho)  # regras do Produto
        if manga not in self.MANGAS:
            raise ValueError(f"manga inválida: {manga}")
        self.manga = manga

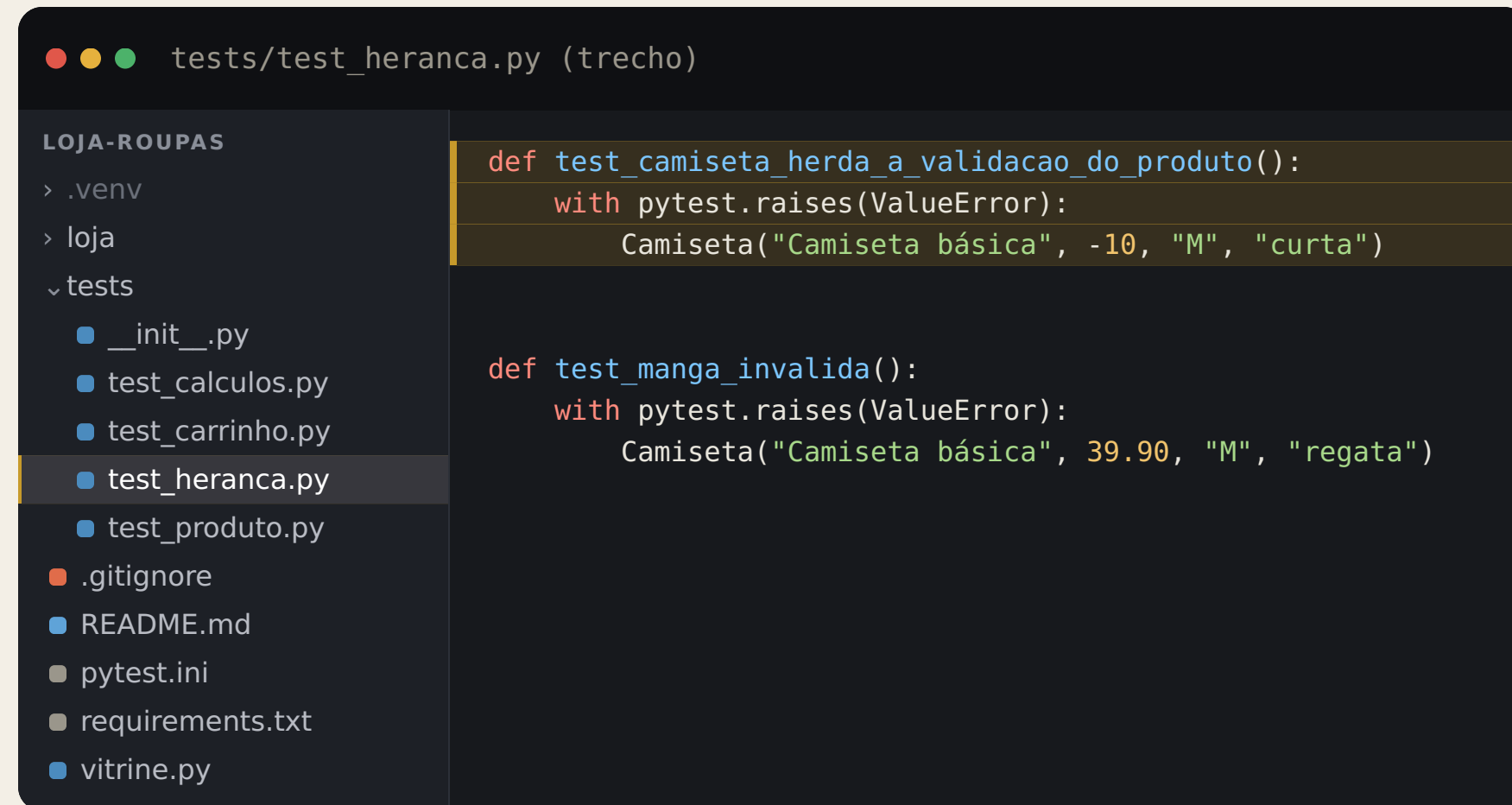
    def descricao(self):
        return f"{super().descricao()} · manga {self.manga}"

class Calca(Produto):
    def __init__(self, nome, preco, tamanho, modelagem):
        super().__init__(nome, preco, tamanho)
        self.modelagem = modelagem

    def descricao(self):
        return f"{super().descricao()} · {self.modelagem}"
```

- ◆ `super().__init__` entrega para o Produto o trabalho comum: atributos e validações.
- ◆ A filha acrescenta só o que é dela, com a própria regra (a manga).
- ◆ `descricao` é sobrescrita, mas reaproveita a da mãe com `super()`.

O teste que prova o super()



```
tests/test_heranca.py (trecho)

LOJA-ROUPAS
> .venv
> loja
  tests
    __init__.py
    test_calculos.py
    test_carrinho.py
    test_heranca.py
    test_produto.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

def test_camiseta_herda_a_validacao_do_produto():
    with pytest.raises(ValueError):
        Camiseta("Camiseta básica", -10, "M", "curta")

def test_manga_invalida():
    with pytest.raises(ValueError):
        Camiseta("Camiseta básica", 39.90, "M", "regata")
```

- ◆ Se alguém apagar a linha do `super()`, a Camiseta aceitaria preço -10. Este teste pega.
- ◆ `isinstance` confirma: onde se espera um Produto, uma Camiseta serve. O carrinho aceita.

FASE 2-3 • CONCLUÍDA

Checkpoint: salvar e subir

```
checkpoint  
  
$ pytest  
19 passed  
$ git add .  
$ git commit -m "Fase 2-3: Camiseta e Calca"  
$ git push
```

CONFIRA

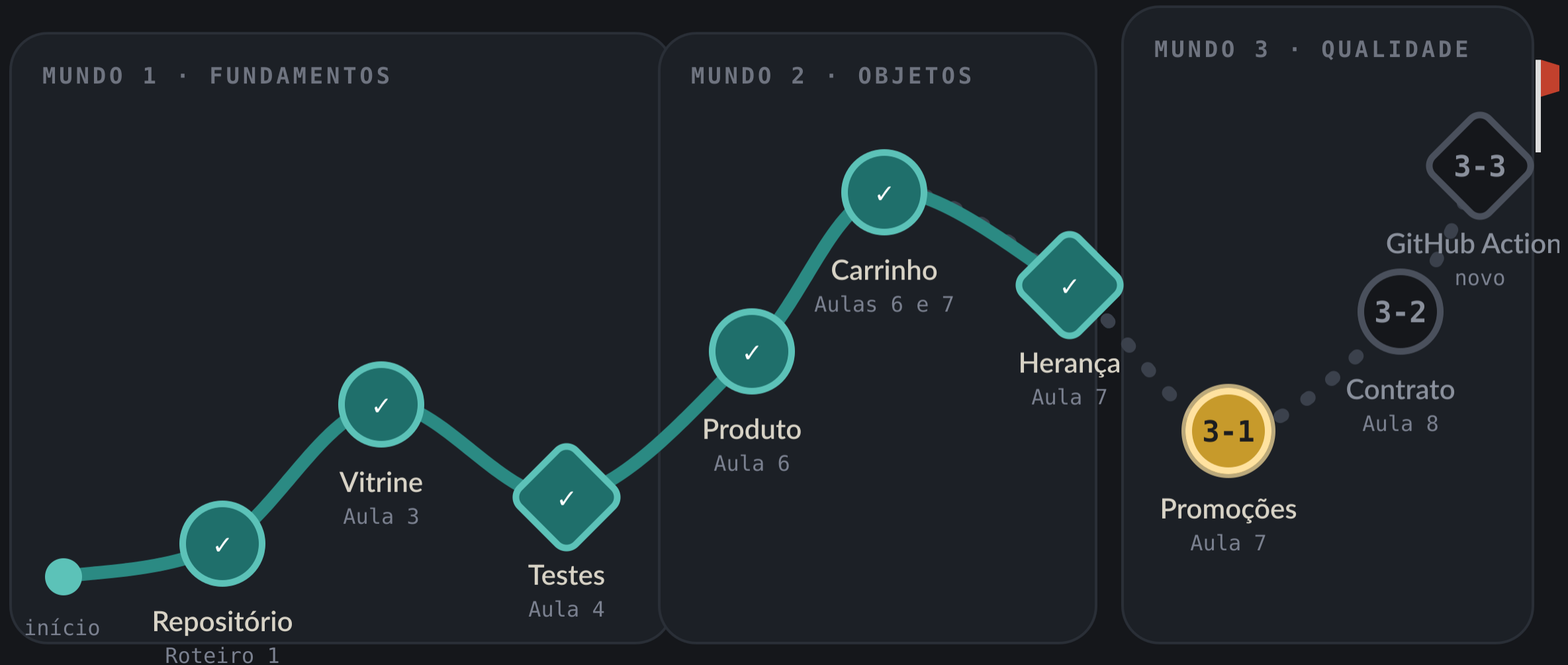
O `pytest` mostra **19 passed**.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-06.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-06`



FASE 3-1 · REVISITA AULA 7

Promoções

Um verbo, várias formas: o carrinho aplica a promoção sem saber qual é.

AO FINAL DA FASE

24 testes passando, com um teste rodando para todas as promoções

Uma cadeia de ifs ou objetos trocáveis?

SEM POLIMORFISMO

O carrinho decide tudo

```
loja/carrinho.py

if tipo == "percentual":
    ...
elif tipo == "cupom":
    ...
elif tipo == "black_friday": # mais um
```

Cada promoção nova obriga a mexer no carrinho.

COM POLIMORFISMO

Cada promoção sabe se aplicar

```
loja/carrinho.py

self.promocao.aplicar(self.subtotal)
```

Promoção nova é classe nova. O carrinho não muda uma linha.

Polimorfismo

Objetos diferentes respondem ao **mesmo método**, cada um do seu jeito. Quem chama não precisa saber qual é qual.

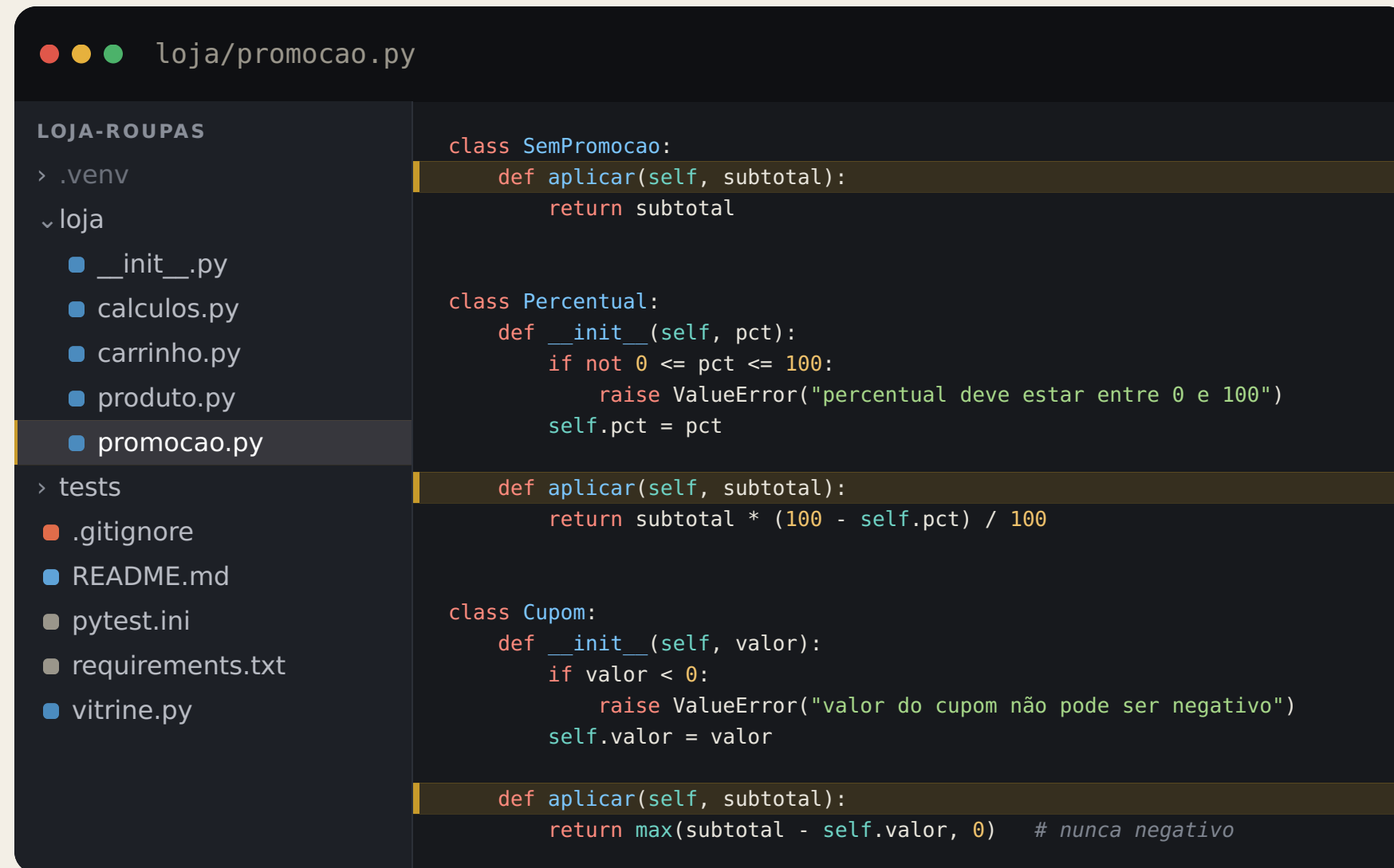
PENSE ASSIM

O caixa só pede "aplique o desconto". Cada promoção sabe fazer a sua conta.

exemplo

```
for promo in [SemPromocao(), Percentual(10), Cupom(50)]:  
    print(promo.aplicar(200))  
# 200  
# 180.0  
# 150
```

loja/promocao.py: três promoções, um verbo



```
class SemPromocao:
    def aplicar(self, subtotal):
        return subtotal

class Percentual:
    def __init__(self, pct):
        if not 0 <= pct <= 100:
            raise ValueError("percentual deve estar entre 0 e 100")
        self.pct = pct

    def aplicar(self, subtotal):
        return subtotal * (100 - self.pct) / 100

class Cupom:
    def __init__(self, valor):
        if valor < 0:
            raise ValueError("valor do cupom não pode ser negativo")
        self.valor = valor

    def aplicar(self, subtotal):
        return max(subtotal - self.valor, 0)  # nunca negativo
```

- ◆ As três têm `aplicar(subtotal)`: recebe o subtotal, devolve o valor com desconto.
- ◆ Nenhuma herda da outra. O que as une é o **nome do método**: duck typing.
- ◆ Cada uma protege a própria regra: percentual entre 0 e 100, cupom nunca negativo.

Parâmetro com valor padrão

Um parâmetro pode ter um valor que vale **quando ninguém informa**. Assim, as chamadas antigas continuam funcionando.

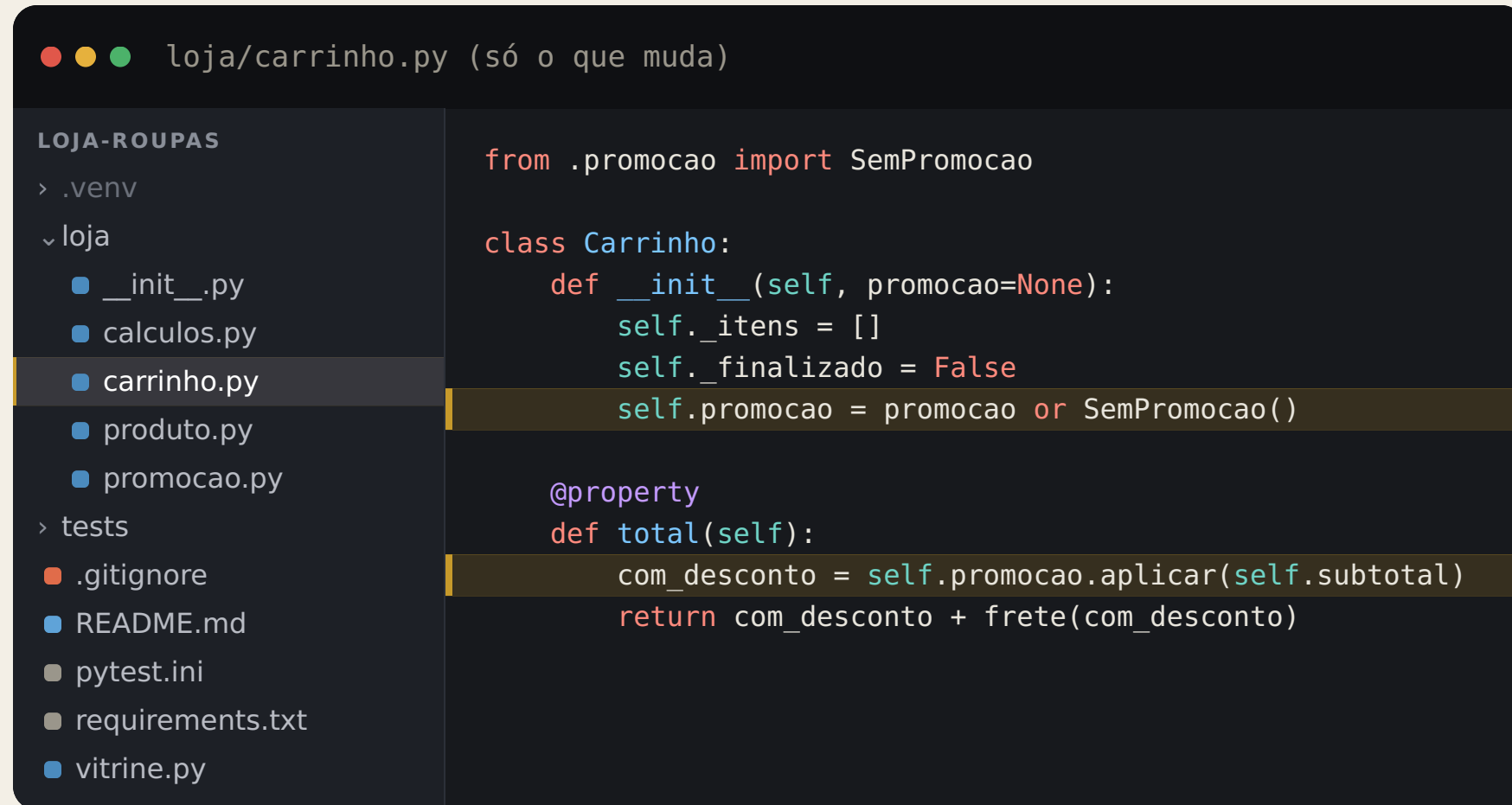
CUIDADO

O valor padrão vem sempre por último na lista de parâmetros.

exemplo

```
def adicionar(self, produto, quantidade=1):  
    ...  
  
c.adicionar(camiseta)           # quantidade = 1  
c.adicionar(camiseta, 3)        # quantidade = 3  
  
promocao or SemPromocao()      # se promocao for None, usa SemPromocao()
```

O carrinho TEM uma promoção



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'LOJA-ROUPAS' with files like __init__.py, calculos.py, carrinho.py, produto.py, promocao.py, .gitignore, README.md, pytest.ini, requirements.txt, and vitrine.py. The code editor shows the implementation of the Carrinho class in carrinho.py, including imports, class definition, __init__ method, and a total property method.

```
loja/carrinho.py (só o que muda)

LOJA-ROUPAS
> .venv
└─ loja
   ├── __init__.py
   ├── calculos.py
   └── carrinho.py
       ├── produto.py
       └── promocao.py
> tests
   ├── .gitignore
   ├── README.md
   ├── pytest.ini
   ├── requirements.txt
   └── vitrine.py

from .promocao import SemPromocao

class Carrinho:
    def __init__(self, promocao=None):
        self._itens = []
        self._finalizado = False
        self.promocao = promocao or SemPromocao()

    @property
    def total(self):
        com_desconto = self.promocao.aplicar(self.subtotal)
        return com_desconto + frete(com_desconto)
```

- ◆ Sem promoção, entra a `SemPromocao`: o código do total não precisa de `if`.
- ◆ O frete é calculado **depois** do desconto. Regra de negócio, decidida pela loja.
- ◆ Os 15 testes da Fase 2-2 continuam passando: `Carrinho()` sem argumento ainda funciona.

parametrize

Roda o mesmo teste várias vezes, uma para cada linha de dados. Menos código repetido, mais casos testados.

CUIDADO

Os nomes dentro das aspas precisam ser os mesmos dos parâmetros do teste.

exemplo

```
@pytest.mark.parametrize("valor, esperado", [
    (199.99, 15.0),
    (200.00, 0.0),
])
def test_frete(valor, esperado):
    assert frete(valor) == esperado
```

Um teste, três rodadas: parametrize

```
tests/test_promocao.py (trecho)

LOJA-ROUPAS
> .venv
> loja
> tests
  ├── __init__.py
  ├── test_calculos.py
  ├── test_carrinho.py
  ├── test_heranca.py
  ├── test_produto.py
  └── test_promocao.py
  ├── .gitignore
  ├── README.md
  ├── pytest.ini
  ├── requirements.txt
  └── vitrine.py

def carrinho_com(promocao):
    c = Carrinho(promocao)
    c.adicionar(Produto("Camiseta básica", 39.90, "M"), 3)
    c.adicionar(Produto("Calça jeans", 129.90, "G"))
    return c # subtotal 249.60

@pytest.mark.parametrize("promocao, esperado", [
    (SemPromocao(), 249.60), # frete grátis
    (Percentual(10), 224.64), # ainda acima de 200
    (Cupom(100), 164.60), # 149.60 + 15 de frete
])
def test_total_com_cada_promocao(promocao, esperado):
    assert carrinho_com(promocao).total == pytest.approx(esperado)
```

- ◆ `parametrize` roda o mesmo teste uma vez para cada linha da lista.
- ◆ No `pytest -v` aparecem três testes:
`[promocao0-249.6]`,
`[promocao1-224.64]`,
`[promocao2-164.6]`.
- ◆ Promoção nova? Uma linha a mais na lista.

Por que o Cupom(100) dá 164.60?

o carrinho do teste

```
3 camisetas x 39.90 = 119.70
1 calça      x 129.90 = 129.90
subtotal                    = 249.60
Cupom(100)                  -> ?
```

$249.60 - 100 = 149.60$.

Como ficou abaixo de 200, o frete volta: +
15.

Total: **164.60**. O teste conferiu uma regra
que ninguém lembraria de checar no olho.

FASE 3-1 • CONCLUÍDA

Checkpoint: salvar e subir

```
checkpoint  
  
$ pytest  
24 passed  
$ git add .  
$ git commit -m "Fase 3-1: promocoos"  
$ git push
```

CONFIRA

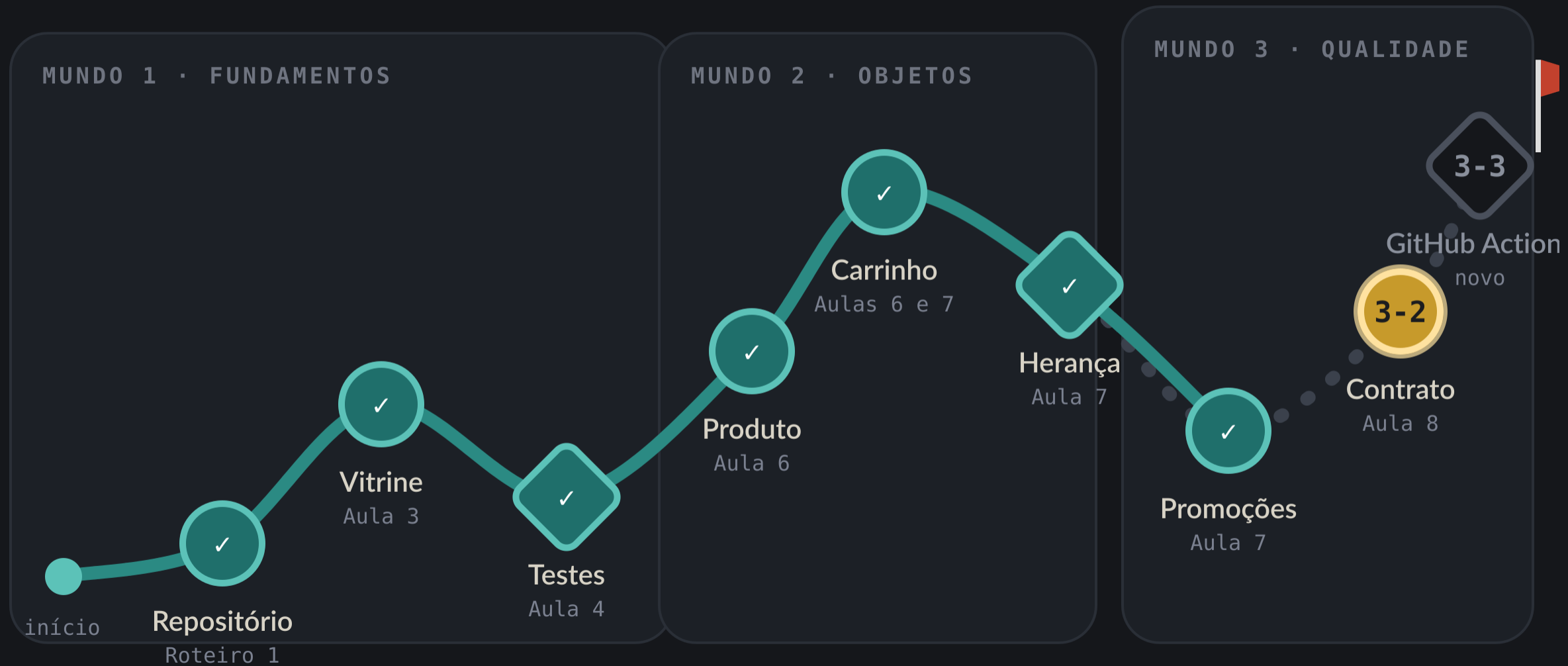
O `pytest -v` mostra os três `test_total_com_cada_promocao[...]` e 24 passed.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-07.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-07`



FASE 3-2 · REVISITA AULA 8 (HOJE)

O contrato

O combinado "toda promoção tem aplicar" vira código, com classe abstrata.

AO FINAL DA FASE

30 testes passando, com promoção fora do contrato barrada

A Black Friday com o nome errado

```
experimento.py

LOJA-ROUPAS
> .venv
> loja
> tests
  .gitignore
  README.md
  experimento.py
  pytest.ini
  requirements.txt
  vitrine.py

from loja.carrinho import Carrinho
from loja.produto import Produto

class BlackFriday:
    def aplicar_desconto(self, subtotal): # nome errado
        return subtotal * 0.5

c = Carrinho(BlackFriday())
c.adicionar(Produto("Moletom", 159.90, "P"))
print("carrinho montado") # aparece
print(c.total)
```

- ◆ O carrinho nasce, recebe a peça e só quebra no **total**, na hora de pagar:
- ◆ `AttributeError: 'BlackFriday' object has no attribute 'aplicar'`
- ◆ O erro aparece longe de onde nasceu. Com o cliente no caixa.

Classe abstrata

Uma classe que só define o **contrato** e não vira objeto. Toda filha é obrigada a implementar os métodos marcados como abstratos.

PENSE ASSIM

O modelo de contrato da loja: ninguém assina o modelo em branco, mas todo contrato real precisa ter as cláusulas dele.

CUIDADO

O contrato confere o **nome** do método, não se a conta está certa. Isso é papel do teste.

exemplo

```
from abc import ABC, abstractmethod

class Promocao(ABC):
    @abstractmethod
    def aplicar(self, subtotal): ...

Promocao() # TypeError: não vira objeto
```

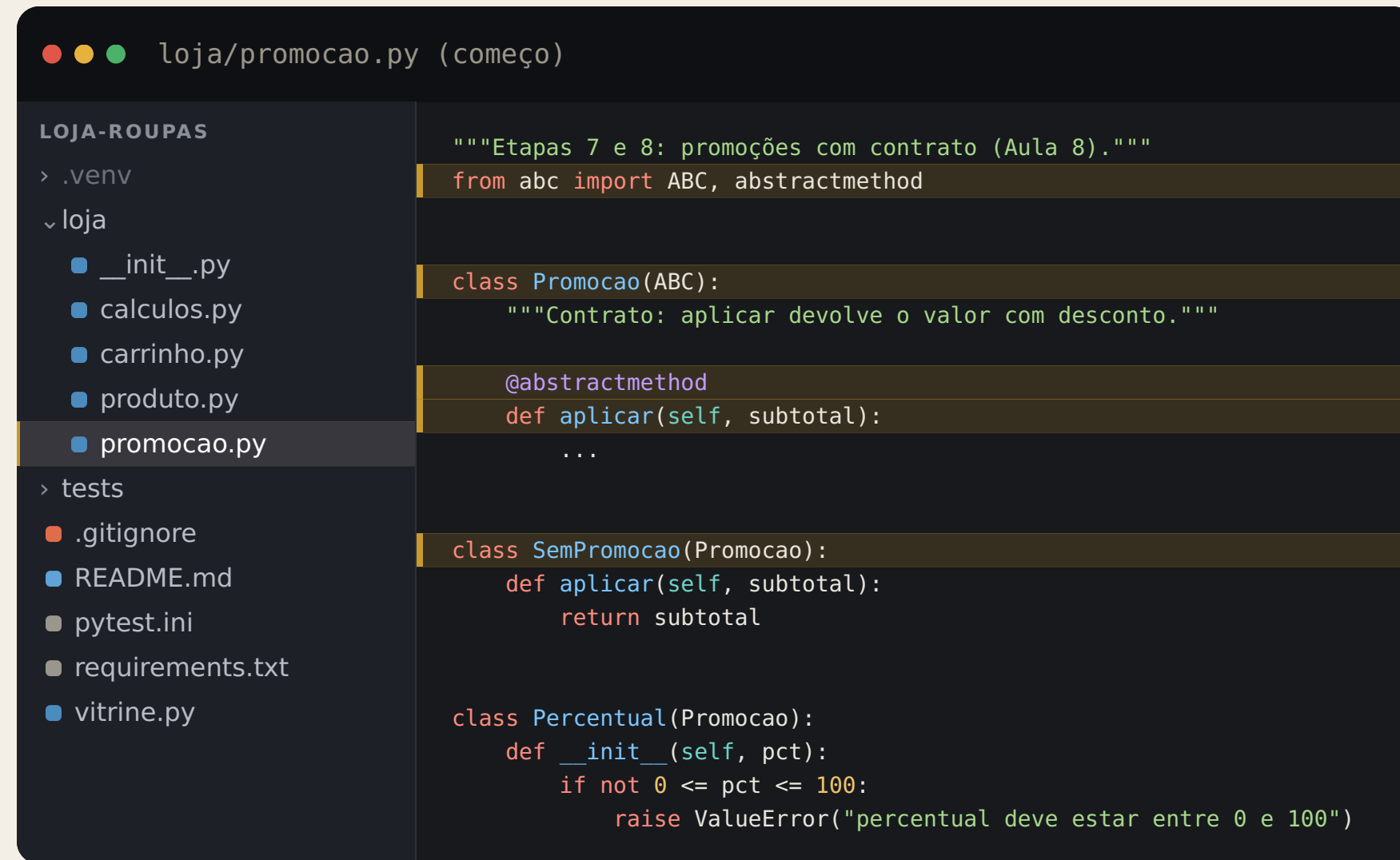
Decorator

A linha com `@` em cima de uma função muda o comportamento dela, sem mexer no corpo. Na loja, já usamos três.

exemplo

```
@property           # lê como atributo
@abstractmethod     # obriga a filha a implementar
@pytest.mark.parametrize # roda o teste várias vezes
```

O contrato escrito: Promocao(ABC)



```
loja/promocao.py (começo)

LOJA-ROUPAS
> .venv
└─ loja
   ├── __init__.py
   ├── calculos.py
   ├── carrinho.py
   ├── produto.py
   └── promocao.py
> tests
   ├── .gitignore
   ├── README.md
   ├── pytest.ini
   ├── requirements.txt
   └── vitrine.py

"""Etapas 7 e 8: promoções com contrato (Aula 8)."""
from abc import ABC, abstractmethod

class Promocao(ABC):
    """Contrato: aplicar devolve o valor com desconto."""

    @abstractmethod
    def aplicar(self, subtotal):
        ...

class SemPromocao(Promocao):
    def aplicar(self, subtotal):
        return subtotal

class Percentual(Promocao):
    def __init__(self, pct):
        if not 0 <= pct <= 100:
            raise ValueError("percentual deve estar entre 0 e 100")
```

- ◆ `Promocao(ABC)` é uma **classe abstrata**: existe para definir o contrato, não para virar objeto.
- ◆ `@abstractmethod` marca o obrigatório. Subclasse sem `aplicar` não nasce.
- ◆ Percentual e Cupom ganham o mesmo `(Promocao)`. O resto não muda.

Agora o erro aparece ao criar

```
terminal  
  
$ python experimento.py  
TypeError: Can't instantiate abstract class BlackFriday  
  without an implementation for abstract method 'aplicar'  
# Python 3.11 ou anterior: "... with abstract method aplicar"
```

- ◆ Faça a BlackFriday herdar de `Promocao` e rode de novo.
- ◆ O erro saiu do caixa e foi para a linha que **cria** a promoção.
- ◆ O "carrinho montado" nem aparece. Quanto mais cedo, mais barato.

Testes que provam o contrato

```
tests/test_contrato.py (trecho)

LOJA-ROUPAS
> .venv
> loja
  tests
    __init__.py
    test_calculos.py
    test_carrinho.py
    test_contrato.py
    test_heranca.py
    test_produto.py
    test_promocao.py
  .gitignore
  README.md
  pytest.ini
  requirements.txt
  vitrine.py

def test_promocao_sem_aplicar_nao_nasce():
    class BlackFriday(Promocao):
        def aplicar_desconto(self, subtotal): # nome errado
            return subtotal * 0.5

    with pytest.raises(TypeError):
        BlackFriday()

def test_carrinho_recusa_quem_nao_segue_o_contrato():
    class Parecida:
        def aplicar(self, subtotal):
            return subtotal

    with pytest.raises(TypeError):
        Carrinho(Parecida())
```

- ◆ Os testes esperam `TypeError`: o contrato barrando quem não o segue.
- ◆ O carrinho também confere na entrada com `isinstance(promocao, Promocao)`, como o preço na Fase 2-1.
- ◆ O contrato confere o **nome** do método. Se ele calcula certo, quem confere é o teste.

FASE 3-2 • CONCLUÍDA

Checkpoint: salvar e subir

```
checkpoint  
  
$ pytest  
30 passed  
$ git add .  
$ git commit -m "Fase 3-2: contrato Promocao"  
$ git push
```

CONFIRA

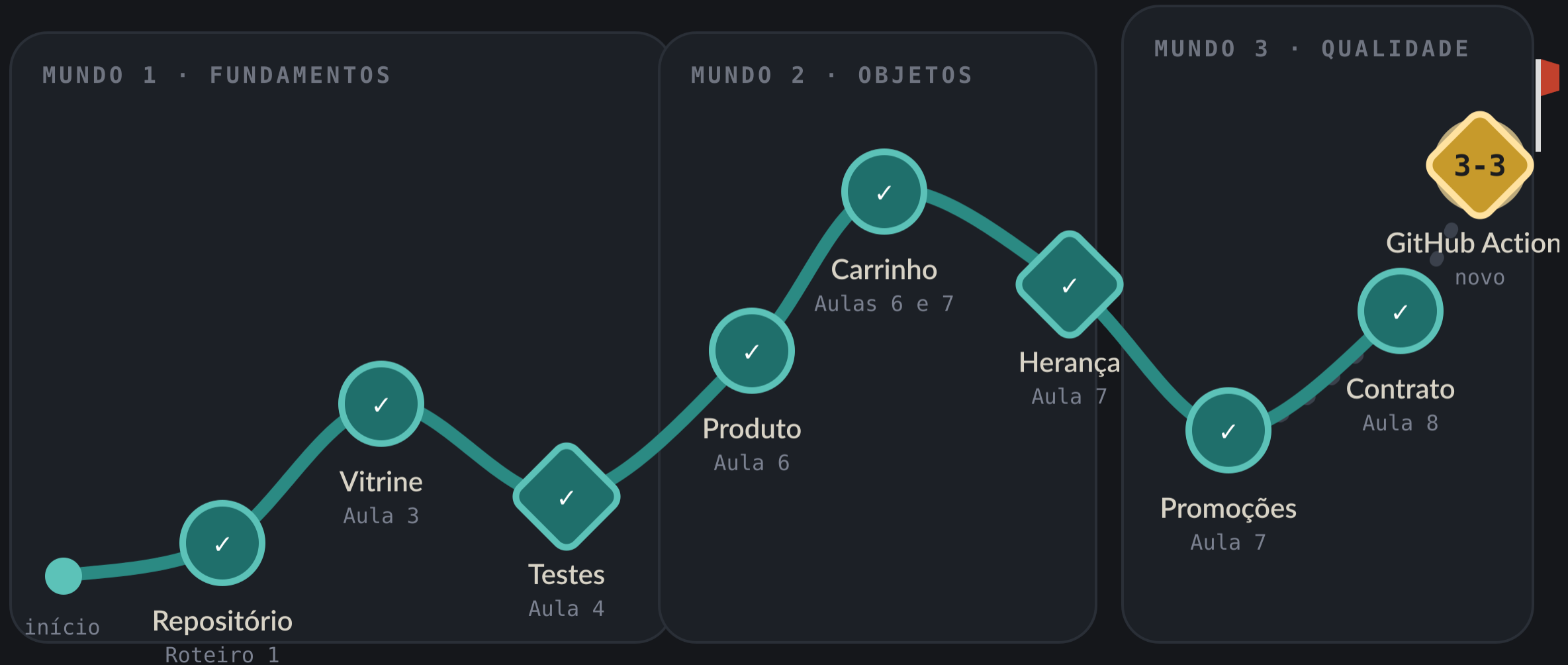
O `pytest` mostra 30 passed.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-08.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-08`



FASE 3-3 · CONTEÚDO NOVO

O GitHub roda os testes

O último chefe: integração contínua. A partir daqui, ninguém da equipe consegue esquecer de rodar os testes.

AO FINAL DA FASE

✓ verde no GitHub a cada push, sem ninguém rodar nada

Integração contínua: o pytest roda sozinho a cada push



✓ VERDE

Todos passaram

Aparece ao lado do commit. A equipe sabe que o código no GitHub funciona.

✗ VERMELHO

Algun falhou

O GitHub avisa por e-mail e mostra qual teste quebrou, antes de alguém usar o código.

RELEMBRE

CONTEÚDO NOVO

YAML

Formato de texto para **configuração**, organizado por indentação: chave, dois pontos, valor. Itens de lista começam com hífen.

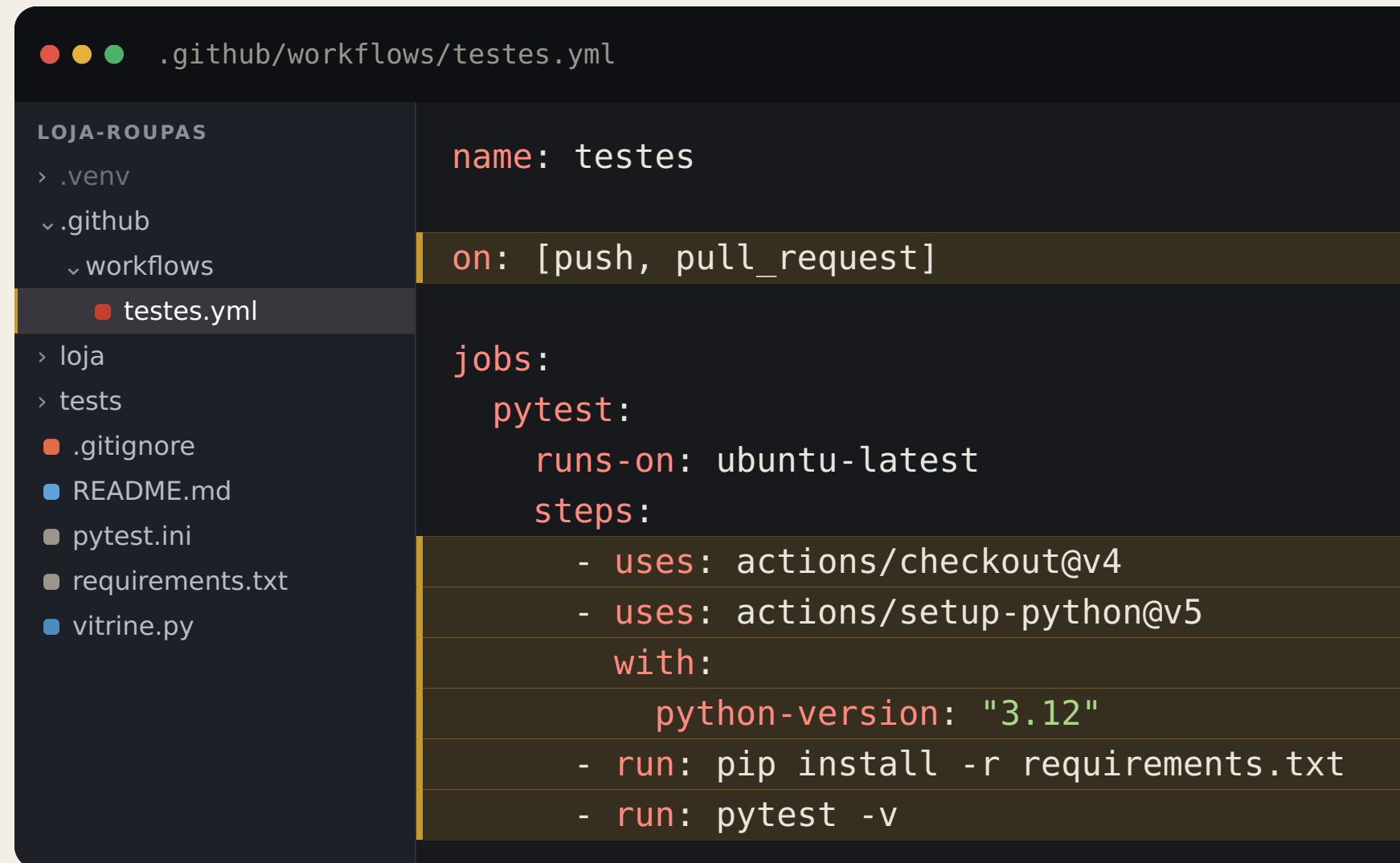
CUIDADO

Indentação com espaços, nunca com tab. Um espaço a mais muda o sentido.

exemplo

```
nome: testes           # chave: valor
on: [push]             # lista numa linha
steps:                 # lista em várias linhas
  - run: pytest -v
  - run: echo pronto
```

.github/workflows/testes.yml



```
.github/workflows/testes.yml

name: testes

on: [push, pull_request]

jobs:
  pytest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest -v
```

- ◆ Crie as pastas `.github/workflows/`, com ponto no início. O nome do arquivo é livre.
- ◆ `on: [push, pull_request]`: quando rodar.
- ◆ `steps`: baixar o código, instalar o Python, instalar o `requirements.txt` e rodar o `pytest`.
- ◆ Por isso o `requirements.txt` da Fase 1-1 importa: a máquina do GitHub começa vazia.

Ver o ✓ aparecer

- ◆ `git add .`, `git commit -m "Fase 3-3: GitHub Actions"` e `git push`.
- ◆ No GitHub, abra a aba **Actions**: o workflow **testes** aparece rodando, em amarelo.
- ◆ Em menos de um minuto: ✓ verde. Clique nele e veja a saída do `pytest`, igual à do seu terminal.
- ◆ Na página inicial do repositório, o ✓ aparece ao lado do último commit.

O SELO NO README

Troque SEU-USUARIO e cole no README.md:

```
![testes](https://github.com/SEU-USUARIO/loja-roupas/actions/workflows/testes.yml/badge.svg)
```

FASE 3-3 · QUEBRE DE PROPÓSITO

Mude o frete grátis para R\$ 300 e dê push.

O GitHub fica vermelho e aponta o teste `test_frete_a_partir_de_200_e_gratis`. Desfaça, dê push de novo e veja voltar o verde. É assim que uma equipe trabalha sem medo de mexer no código.

Checkpoint: salvar e subir

```
checkpoint  
  
$ pytest  
30 passed  
$ git add .  
$ git commit -m "Fase 3-3: GitHub Actions"  
$ git push
```

CONFIRA

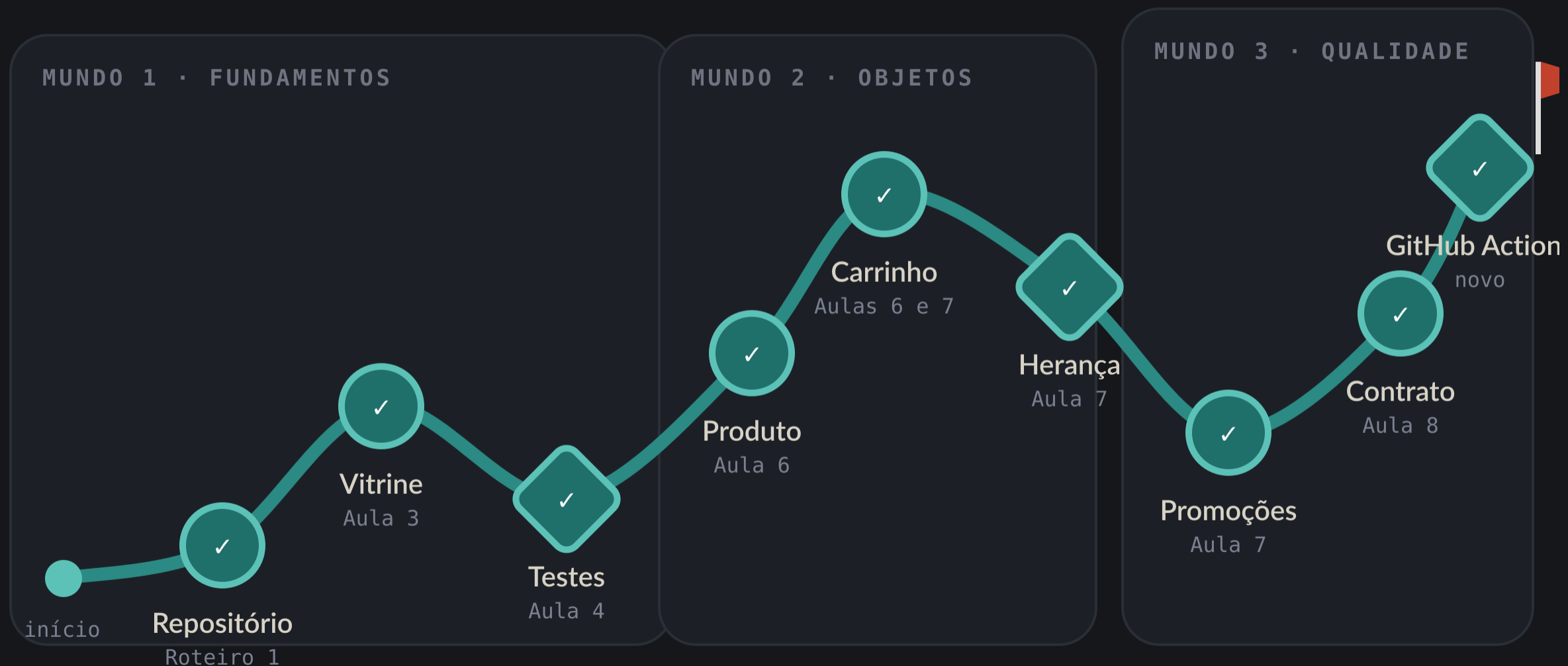
A aba **Actions** mostra o workflow testes com ✓ verde, e o selo aparece no README.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-09.zip

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-09`



O MAPA DE HOJE

Mapa completo

Do repositório vazio a um projeto testado, com integração contínua. Tudo o que vimos desde a Aula 1, num projeto só.

FECHAMENTO

O que cada fase revisitou

FASE	CONCEITO	AULA	TESTE QUE PROVA
1-1	Git, GitHub, venv, .gitignore	Roteiro 1	o repositório no ar
1-2	tupla, lista, dicionário	Aula 3	o script imprime 249.6
1-3	função, módulo, assert, pytest, teste primeiro	Aula 4	<code>test_frete_a_partir_de_200_e_gratis</code>
2-1	classe e invariante	Aula 6	<code>test_preco_zero_nao_e_aceito</code>
2-2	composição, encapsulamento, @property	Aulas 6 e 7	<code>test_itens_devolve_copia</code>
2-3	herança, super(), sobrescrita	Aula 7	<code>test_camiseta_herda_a_validacao_do_produto</code>
3-1	polimorfismo, parametrize	Aula 7	<code>test_total_com_cada_promocao</code>
3-2	classe abstrata, contrato	Aula 8	<code>test_promocao_sem_aplicar_nao_nasce</code>
3-3	integração contínua	novo	✓ no GitHub

Checklist para levar tudo isso ao projeto de vocês

- ◆ Repositório da equipe com `.gitignore`, `requirements.txt` e README com como rodar os testes.
- ◆ Regras de negócio em **funções** ou **classes** que validam na entrada, cada uma com um teste que espera o erro.
- ◆ Pelo menos um teste escrito **antes** do código, para uma regra nova.
- ◆ Um verbo com duas implementações e um **contrato** com classe abstrata.
- ◆ O workflow do **GitHub Actions** copiado da loja, com ✓ verde.
- ◆ Commits pequenos, com mensagem que diz o que mudou.

PARA DEIXAR ABERTO

Cola de comandos

git

```
# pergunta o que mudou
git status
# prepara tudo para o commit
git add .
# tira a foto
git commit -m "mensagem"
# envia para o GitHub
git push
# traz o que a equipe enviou
git pull
```

python e pytest

```
# criar e ativar o ambiente
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt
# rodar os testes
pytest
pytest -v
pytest tests/test_carrinho.py
```